File Tree
해시 확인: certutil -hashfile <dist> SHA256
dist/
├── index.html (77433 B / 8ab941e91919)
├── main.js (33823 B / 04a8e344941e)
└── style.css (14083 B / 3bb92984b4f4)

==== index.html (77433 B / 8ab941e91919) ====

```
<!doctype html>
<html lang="ko">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>TS Chess</title>
    <script id="ts-chess-shared-core-source" type="text/plain">
      var TSChessShared = (function (exports) {
        'use strict';
        const MASK64 = 0xffffffffffffffffn;
        function rotl(x, k) {
          return ((x << k) | (x >> (64n - k))) & 0xffffffffffffffffn;
        }
        function wrappingMul(x, y) {
          return (x * y) & MASK64;
        }
        function xoroshiro128(state) {
          return function () {
            let s0 = BigInt(state & MASK64);
            let s1 = BigInt((state >> 64n) & MASK64);
            const result = wrappingMul(rotl(wrappingMul(s0, 5n), 7n), 9n);
            s1 ^= s0;
            s0 = (rotl(s0, 24n) ^ s1 ^ (s1 << 16n)) & MASK64;
            s1 = rotl(s1, 37n);
            state = (s1 << 64n) | s0;
            return result;
          };
        }
        const rand = xoroshiro128(0xa187eb39cdcaed8f31c4b365b102e01en);
        const PIECE_KEYS = Array.from({ length: 2 }, () =>
          Array.from({ length: 6 }, () => Array.from({ length: 128 }, () => rand())),
        );
        const EP_KEYS = Array.from({ length: 8 }, () => rand());
        const CASTLING_KEYS = Array.from({ length: 16 }, () => rand());
        const SIDE_KEY = rand();
        const WHITE = 'w';
        const BLACK = 'b';
        const PAWN = 'p';
        const KNIGHT = 'n';
        const BISHOP = 'b';
        const ROOK = 'r';
        const QUEEN = 'q';
        const KING = 'k';
        const DEFAULT_POSITION = 'rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1';
        class Move {
          color;
          from;
          to;
          piece;
          captured;
          promotion;
          /**
           * @deprecated This field is deprecated and will be removed in version 2.0.0.
           * Please use move descriptor functions instead: `isCapture`, `isPromotion`,
           * `isEnPassant`, `isKingsideCastle`, `isQueensideCastle`, `isCastle`, and
           * `isBigPawn`
           */
          flags;
          san;
          lan;
          before;
```

```
    after;
    constructor(chess, internal) {
      const { color, piece, from, to, flags, captured, promotion } = internal;
      const fromAlgebraic = algebraic(from);
      const toAlgebraic = algebraic(to);
      this.color = color;
      this.piece = piece;
      this.from = fromAlgebraic;
      this.to = toAlgebraic;
      this.san = chess['_moveToSan'](internal, chess['_moves']({ legal: true }));
      this.lan = fromAlgebraic + toAlgebraic;
      this.before = chess.fen();
      chess['_makeMove'](internal);
      this.after = chess.fen();
      chess['_undoMove']();
      this.flags = '';
      for (const flag in BITS) {
        if (BITS[flag] & flags) {
          this.flags += FLAGS[flag];
        }
      }
      if (captured) {
        this.captured = captured;
      }
      if (promotion) {
        this.promotion = promotion;
        this.lan += promotion;
      }
    }
    isCapture() {
      return this.flags.indexOf(FLAGS['CAPTURE']) > -1;
    }
    isPromotion() {
      return this.flags.indexOf(FLAGS['PROMOTION']) > -1;
    }
    isEnPassant() {
      return this.flags.indexOf(FLAGS['EP_CAPTURE']) > -1;
    }
    isKingsideCastle() {
      return this.flags.indexOf(FLAGS['KSIDE_CASTLE']) > -1;
    }
    isQueensideCastle() {
      return this.flags.indexOf(FLAGS['QSIDE_CASTLE']) > -1;
    }
    isBigPawn() {
      return this.flags.indexOf(FLAGS['BIG_PAWN']) > -1;
    }
  }
  const EMPTY = -1;
  const FLAGS = {
    NORMAL: 'n',
    CAPTURE: 'c',
    BIG_PAWN: 'b',
    EP_CAPTURE: 'e',
    PROMOTION: 'p',
    KSIDE_CASTLE: 'k',
    QSIDE_CASTLE: 'q',
    NULL_MOVE: '-',
  };
  const SQUARES = [
    'a8',
    'b8',
    'c8',
    'd8',
    'e8',
    'f8',
    'g8',
    'h8',
    'a7',
    'b7',
    'c7',
```

```
        'd7',
        'e7',
        'f7',
        'g7',
        'h7',
        'a6',
        'b6',
        'c6',
        'd6',
        'e6',
        'f6',
        'g6',
        'h6',
        'a5',
        'b5',
        'c5',
        'd5',
        'e5',
        'f5',
        'g5',
        'h5',
        'a4',
        'b4',
        'c4',
        'd4',
        'e4',
        'f4',
        'g4',
        'h4',
        'a3',
        'b3',
        'c3',
        'd3',
        'e3',
        'f3',
        'g3',
        'h3',
        'a2',
        'b2',
        'c2',
        'd2',
        'e2',
        'f2',
        'g2',
        'h2',
        'a1',
        'b1',
        'c1',
        'd1',
        'e1',
        'f1',
        'g1',
        'h1',
    ];
    const BITS = {
        NORMAL: 1,
        CAPTURE: 2,
        BIG_PAWN: 4,
        EP_CAPTURE: 8,
        PROMOTION: 16,
        KSIDE_CASTLE: 32,
        QSIDE_CASTLE: 64,
        NULL_MOVE: 128,
    };
    const Ox88 = {
        a8: 0,
        b8: 1,
        c8: 2,
        d8: 3,
        e8: 4,
        f8: 5,
```

```
        g8: 6,
        h8: 7,
        a7: 16,
        b7: 17,
        c7: 18,
        d7: 19,
        e7: 20,
        f7: 21,
        g7: 22,
        h7: 23,
        a6: 32,
        b6: 33,
        c6: 34,
        d6: 35,
        e6: 36,
        f6: 37,
        g6: 38,
        h6: 39,
        a5: 48,
        b5: 49,
        c5: 50,
        d5: 51,
        e5: 52,
        f5: 53,
        g5: 54,
        h5: 55,
        a4: 64,
        b4: 65,
        c4: 66,
        d4: 67,
        e4: 68,
        f4: 69,
        g4: 70,
        h4: 71,
        a3: 80,
        b3: 81,
        c3: 82,
        d3: 83,
        e3: 84,
        f3: 85,
        g3: 86,
        h3: 87,
        a2: 96,
        b2: 97,
        c2: 98,
        d2: 99,
        e2: 100,
        f2: 101,
        g2: 102,
        h2: 103,
        a1: 112,
        b1: 113,
        c1: 114,
        d1: 115,
        e1: 116,
        f1: 117,
        g1: 118,
        h1: 119,
    };
    const PAWN_OFFSETS = {
        b: [16, 32, 17, 15],
        w: [-16, -32, -17, -15],
    };
    const PIECE_OFFSETS = {
        n: [-18, -33, -31, -14, 18, 33, 31, 14],
        b: [-17, -15, 17, 15],
        r: [-16, 1, 16, -1],
        q: [-17, -16, -15, 1, 17, 16, 15, -1],
        k: [-17, -16, -15, 1, 17, 16, 15, -1],
    };
    const ATTACKS = [
```

```
        20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20,
        0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 24, 0, 0, 0, 20,
        0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 2, 24, 2, 20,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 24, 24, 24, 24, 24, 24, 56,
        0, 56, 24, 24, 24, 24, 24, 24, 0, 0, 0, 0, 0, 0, 2, 53, 56, 53, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        0, 20, 2, 24, 2, 20, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 20, 0, 0, 24, 0, 0, 20, 0, 0, 0, 0, 0, 0, 0,
        0, 20, 0, 0, 0, 24, 0, 0, 0, 20, 0, 0, 0, 0, 0, 0, 20, 0, 0, 0, 0, 24, 0, 0, 0, 0, 20, 0, 0, 0,
        0, 20, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 20, 0, 0, 20, 0, 0, 0, 0, 0, 0, 24, 0, 0, 0, 0, 0, 0,
        20,
    ];
    const RAYS = [
        17, 0, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 0, 15, 0, 0, 17, 0, 0, 0, 0, 0, 16, 0, 0, 0, 0, 0, 15,
        0, 0, 0, 0, 17, 0, 0, 0, 0, 16, 0, 0, 0, 0, 15, 0, 0, 0, 0, 0, 0, 17, 0, 0, 0, 16, 0, 0, 0, 15,
        0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 0, 16, 0, 0, 15, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 0, 16, 0, 15,
        0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 17, 16, 15, 0, 0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 0, -1,
        -1, -1, -1, -1, -1, -1, -1, 0, 0, 0, 0, 0, 0, 0, -15, -16, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
        -15, 0, -16, 0, -17, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, -15, 0, 0, -16, 0, 0, -17, 0, 0, 0, 0, 0, 0,
        0, 0, -15, 0, 0, 0, -16, 0, 0, 0, -17, 0, 0, 0, 0, 0, 0, -15, 0, 0, 0, 0, -16, 0, 0, 0, 0, -17,
        0, 0, 0, 0, -15, 0, 0, 0, 0, 0, -16, 0, 0, 0, 0, 0, -17, 0, 0, -15, 0, 0, 0, 0, 0, 0, -16, 0, 0,
        0, 0, 0, 0, -17,
    ];
    const PIECE_MASKS = { p: 1, n: 2, b: 4, r: 8, q: 16, k: 32 };
    const SYMBOLS = 'pnbrqkPNBRQK';
    const PROMOTIONS = [KNIGHT, BISHOP, ROOK, QUEEN];
    const RANK_1 = 7;
    const RANK_2 = 6;
    const RANK_7 = 1;
    const RANK_8 = 0;
    const ROOKS = {
        w: [
            { square: Ox88.a1, flag: BITS.QSIDE_CASTLE },
            { square: Ox88.h1, flag: BITS.KSIDE_CASTLE },
        ],
        b: [
            { square: Ox88.a8, flag: BITS.QSIDE_CASTLE },
            { square: Ox88.h8, flag: BITS.KSIDE_CASTLE },
        ],
    };
    const SECOND_RANK = { b: RANK_7, w: RANK_2 };
    function rank(square) {
        return square >> 4;
    }
    function file(square) {
        return square & 15;
    }
    function isDigit(c) {
        return '0123456789'.indexOf(c) !== -1;
    }
    function algebraic(square) {
        const f = file(square);
        const r = rank(square);
        return 'abcdefgh'.substring(f, f + 1) + '87654321'.substring(r, r + 1);
    }
    function swapColor(color) {
        return color === WHITE ? BLACK : WHITE;
    }
    function validateFen(fen) {
        const tokens = fen.split(/\s+/);
        if (tokens.length !== 6) {
            return {
                ok: false,
                error: 'Invalid FEN: must contain six space-delimited fields',
            };
        }
        const moveNumber = parseInt(tokens[5], 10);
        if (isNaN(moveNumber) || moveNumber <= 0) {
            return {
                ok: false,
                error: 'Invalid FEN: move number must be a positive integer',
            };
        }
```

```
    const halfMoves = parseInt(tokens[4], 10);
    if (isNaN(halfMoves) || halfMoves < 0) {
      return {
        ok: false,
        error: 'Invalid FEN: half move counter number must be a non-negative integer',
      };
    }
    if (!/^(-|[abcdefgh][36])$/.test(tokens[3])) {
      return { ok: false, error: 'Invalid FEN: en-passant square is invalid' };
    }
    if (/[^kKqQ-]/.test(tokens[2])) {
      return { ok: false, error: 'Invalid FEN: castling availability is invalid' };
    }
    if (!/^(w|b)$/.test(tokens[1])) {
      return { ok: false, error: 'Invalid FEN: side-to-move is invalid' };
    }
    const rows = tokens[0].split('/');
    if (rows.length !== 8) {
      return {
        ok: false,
        error: "Invalid FEN: piece data does not contain 8 '/'-delimited rows",
      };
    }
    for (let i = 0; i < rows.length; i++) {
      let sumFields = 0;
      let previousWasNumber = false;
      for (let k = 0; k < rows[i].length; k++) {
        if (isDigit(rows[i][k])) {
          if (previousWasNumber) {
            return {
              ok: false,
              error: 'Invalid FEN: piece data is invalid (consecutive number)',
            };
          }
          sumFields += parseInt(rows[i][k], 10);
          previousWasNumber = true;
        } else {
          if (!/^[prnbqkPRNBQK]$/.test(rows[i][k])) {
            return {
              ok: false,
              error: 'Invalid FEN: piece data is invalid (invalid piece)',
            };
          }
          sumFields += 1;
          previousWasNumber = false;
        }
      }
      if (sumFields !== 8) {
        return {
          ok: false,
          error: 'Invalid FEN: piece data is invalid (too many squares in rank)',
        };
      }
    }
    if ((tokens[3][1] === '3' && tokens[1] === 'w') || (tokens[3][1] === '6' && tokens[1] === 'b')) {
      return { ok: false, error: 'Invalid FEN: illegal en-passant square' };
    }
    const kings = [
      { color: 'white', regex: /K/g },
      { color: 'black', regex: /k/g },
    ];
    for (const { color, regex } of kings) {
      if (!regex.test(tokens[0])) {
        return { ok: false, error: `Invalid FEN: missing ${color} king` };
      }
      if ((tokens[0].match(regex) || []).length > 1) {
        return { ok: false, error: `Invalid FEN: too many ${color} kings` };
      }
    }
    if (Array.from(rows[0] + rows[7]).some((char) => char.toUpperCase() === 'P')) {
      return {
```

```
        ok: false,
        error: 'Invalid FEN: some pawns are on the edge rows',
      };
    }
    return { ok: true };
  }
  function getDisambiguator(move, moves) {
    const from = move.from;
    const to = move.to;
    const piece = move.piece;
    let ambiguities = 0;
    let sameRank = 0;
    let sameFile = 0;
    for (let i = 0, len = moves.length; i < len; i++) {
      const ambigFrom = moves[i].from;
      const ambigTo = moves[i].to;
      const ambigPiece = moves[i].piece;
      if (piece === ambigPiece && from !== ambigFrom && to === ambigTo) {
        ambiguities++;
        if (rank(from) === rank(ambigFrom)) {
          sameRank++;
        }
        if (file(from) === file(ambigFrom)) {
          sameFile++;
        }
      }
    }
    if (ambiguities > 0) {
      if (sameRank > 0 && sameFile > 0) {
        return algebraic(from);
      } else if (sameFile > 0) {
        return algebraic(from).charAt(1);
      } else {
        return algebraic(from).charAt(0);
      }
    }
    return '';
  }
  function addMove(moves, color, from, to, piece, captured = void 0, flags = BITS.NORMAL) {
    const r = rank(to);
    if (piece === PAWN && (r === RANK_1 || r === RANK_8)) {
      for (let i = 0; i < PROMOTIONS.length; i++) {
        const promotion = PROMOTIONS[i];
        moves.push({
          color,
          from,
          to,
          piece,
          captured,
          promotion,
          flags: flags | BITS.PROMOTION,
        });
      }
    } else {
      moves.push({
        color,
        from,
        to,
        piece,
        captured,
        flags,
      });
    }
  }
  class Chess {
    _board = new Array(128);
    _turn = WHITE;
    _kings = { w: EMPTY, b: EMPTY };
    _epSquare = -1;
    _halfMoves = 0;
    _moveNumber = 0;
```

```
    _history = [];
    _castling = { w: 0, b: 0 };
    _hash = 0n;
    // tracks number of times a position has been seen for repetition checking
    _positionCount = /* @__PURE__ */ new Map();
    constructor(fen = DEFAULT_POSITION, { skipValidation = false } = {}) {
      this.load(fen, { skipValidation });
    }
    clear() {
      this._board = new Array(128);
      this._kings = { w: EMPTY, b: EMPTY };
      this._turn = WHITE;
      this._castling = { w: 0, b: 0 };
      this._epSquare = EMPTY;
      this._halfMoves = 0;
      this._moveNumber = 1;
      this._history = [];
      this._hash = this._computeHash();
      this._positionCount = /* @__PURE__ */ new Map();
    }
    load(fen, { skipValidation = false } = {}) {
      let tokens = fen.split(/\s+/);
      if (tokens.length >= 2 && tokens.length < 6) {
        const adjustments = ['-', '-', '0', '1'];
        fen = tokens.concat(adjustments.slice(-(6 - tokens.length))).join(' ');
      }
      tokens = fen.split(/\s+/);
      if (!skipValidation) {
        const { ok, error } = validateFen(fen);
        if (!ok) {
          throw new Error(error);
        }
      }
      const position = tokens[0];
      let square = 0;
      this.clear();
      for (let i = 0; i < position.length; i++) {
        const piece = position.charAt(i);
        if (piece === '/') {
          square += 8;
        } else if (isDigit(piece)) {
          square += parseInt(piece, 10);
        } else {
          const color = piece < 'a' ? WHITE : BLACK;
          this._put({ type: piece.toLowerCase(), color }, algebraic(square));
          square++;
        }
      }
      this._turn = tokens[1];
      if (tokens[2].indexOf('K') > -1) {
        this._castling.w |= BITS.KSIDE_CASTLE;
      }
      if (tokens[2].indexOf('Q') > -1) {
        this._castling.w |= BITS.QSIDE_CASTLE;
      }
      if (tokens[2].indexOf('k') > -1) {
        this._castling.b |= BITS.KSIDE_CASTLE;
      }
      if (tokens[2].indexOf('q') > -1) {
        this._castling.b |= BITS.QSIDE_CASTLE;
      }
      this._epSquare = tokens[3] === '-' ? EMPTY : Ox88[tokens[3]];
      this._halfMoves = parseInt(tokens[4], 10);
      this._moveNumber = parseInt(tokens[5], 10);
      this._hash = this._computeHash();
      this._incPositionCount();
    }
    fen({ forceEnpassantSquare = false } = {}) {
      let empty = 0;
      let fen = '';
      for (let i = Ox88.a8; i <= Ox88.h1; i++) {
```

```
            if (this._board[i]) {
                if (empty > 0) {
                    fen += empty;
                    empty = 0;
                }
                const { color, type: piece } = this._board[i];
                fen += color === WHITE ? piece.toUpperCase() : piece.toLowerCase();
            } else {
                empty++;
            }
            if ((i + 1) & 136) {
                if (empty > 0) {
                    fen += empty;
                }
                if (i !== 0x88.h1) {
                    fen += '/';
                }
                empty = 0;
                i += 8;
            }
        }
        let castling = '';
        if (this._castling[WHITE] & BITS.KSIDE_CASTLE) {
            castling += 'K';
        }
        if (this._castling[WHITE] & BITS.QSIDE_CASTLE) {
            castling += 'Q';
        }
        if (this._castling[BLACK] & BITS.KSIDE_CASTLE) {
            castling += 'k';
        }
        if (this._castling[BLACK] & BITS.QSIDE_CASTLE) {
            castling += 'q';
        }
        castling = castling || '-';
        let epSquare = '-';
        if (this._epSquare !== EMPTY) {
            if (forceEnpassantSquare) {
                epSquare = algebraic(this._epSquare);
            } else {
                const bigPawnSquare = this._epSquare + (this._turn === WHITE ? 16 : -16);
                const squares = [bigPawnSquare + 1, bigPawnSquare - 1];
                for (const square of squares) {
                    if (square & 136) {
                        continue;
                    }
                    const color = this._turn;
                    if (this._board[square]?.color === color && this._board[square]?.type === PAWN) {
                        this._makeMove({
                            color,
                            from: square,
                            to: this._epSquare,
                            piece: PAWN,
                            captured: PAWN,
                            flags: BITS.EP_CAPTURE,
                        });
                        const isLegal = !this._isKingAttacked(color);
                        this._undoMove();
                        if (isLegal) {
                            epSquare = algebraic(this._epSquare);
                            break;
                        }
                    }
                }
            }
        }
        return [fen, this._turn, castling, epSquare, this._halfMoves, this._moveNumber].join(' ');
    }
    _pieceKey(i) {
        if (!this._board[i]) {
            return 0n;
```

```
        }
        const { color, type } = this._board[i];
        const colorIndex = {
          w: 0,
          b: 1,
        }[color];
        const typeIndex = {
          p: 0,
          n: 1,
          b: 2,
          r: 3,
          q: 4,
          k: 5,
        }[type];
        return PIECE_KEYS[colorIndex][typeIndex][i];
      }
      _epKey() {
        return this._epSquare === EMPTY ? 0n : EP_KEYS[this._epSquare & 7];
      }
      _castlingKey() {
        const index = (this._castling.w >> 5) | (this._castling.b >> 3);
        return CASTLING_KEYS[index];
      }
      _computeHash() {
        let hash = 0n;
        for (let i = 0x88.a8; i <= 0x88.h1; i++) {
          if (i & 136) {
            i += 7;
            continue;
          }
          if (this._board[i]) {
            hash ^= this._pieceKey(i);
          }
        }
        hash ^= this._epKey();
        hash ^= this._castlingKey();
        if (this._turn === 'b') {
          hash ^= SIDE_KEY;
        }
        return hash;
      }
      reset() {
        this.load(DEFAULT_POSITION);
      }
      get(square) {
        return this._board[0x88[square]];
      }
      _set(sq, piece) {
        this._hash ^= this._pieceKey(sq);
        this._board[sq] = piece;
        this._hash ^= this._pieceKey(sq);
      }
      _put({ type, color }, square) {
        if (SYMBOLS.indexOf(type.toLowerCase()) === -1) {
          return false;
        }
        if (!(square in 0x88)) {
          return false;
        }
        const sq = 0x88[square];
        if (type == KING && !(this._kings[color] == EMPTY || this._kings[color] == sq)) {
          return false;
        }
        const currentPieceOnSquare = this._board[sq];
        if (currentPieceOnSquare && currentPieceOnSquare.type === KING) {
          this._kings[currentPieceOnSquare.color] = EMPTY;
        }
        this._set(sq, { type, color });
        if (type === KING) {
          this._kings[color] = sq;
        }
```

```
        return true;
      }
      _clear(sq) {
        this._hash ^= this._pieceKey(sq);
        delete this._board[sq];
      }
      _attacked(color, square, verbose) {
        const attackers = [];
        for (let i = 0x88.a8; i <= 0x88.h1; i++) {
          if (i & 136) {
            i += 7;
            continue;
          }
          if (this._board[i] === void 0 || this._board[i].color !== color) {
            continue;
          }
          const piece = this._board[i];
          const difference = i - square;
          if (difference === 0) {
            continue;
          }
          const index = difference + 119;
          if (ATTACKS[index] & PIECE_MASKS[piece.type]) {
            if (piece.type === PAWN) {
              if (
                (difference > 0 && piece.color === WHITE) ||
                (difference <= 0 && piece.color === BLACK)
              ) {
                if (!verbose) {
                  return true;
                } else {
                  attackers.push(algebraic(i));
                }
              }
              continue;
            }
            if (piece.type === 'n' || piece.type === 'k') {
              if (!verbose) {
                return true;
              } else {
                attackers.push(algebraic(i));
                continue;
              }
            }
            const offset = RAYS[index];
            let j = i + offset;
            let blocked = false;
            while (j !== square) {
              if (this._board[j] != null) {
                blocked = true;
                break;
              }
              j += offset;
            }
            if (!blocked) {
              if (!verbose) {
                return true;
              } else {
                attackers.push(algebraic(i));
                continue;
              }
            }
          }
        }
        if (verbose) {
          return attackers;
        } else {
          return false;
        }
      }
      attackers(square, attackedBy) {
```

```
        if (!attackedBy) {
            return this._attacked(this._turn, Ox88[square], true);
        } else {
            return this._attacked(attackedBy, Ox88[square], true);
        }
    }
    _isKingAttacked(color) {
        const square = this._kings[color];
        return square === -1 ? false : this._attacked(swapColor(color), square);
    }
    hash() {
        return this._hash.toString(16);
    }
    isAttacked(square, attackedBy) {
        return this._attacked(attackedBy, Ox88[square]);
    }
    isCheck() {
        return this._isKingAttacked(this._turn);
    }
    inCheck() {
        return this.isCheck();
    }
    isCheckmate() {
        return this.isCheck() && this._moves().length === 0;
    }
    isStalemate() {
        return !this.isCheck() && this._moves().length === 0;
    }
    isInsufficientMaterial() {
        const pieces = {
            b: 0,
            n: 0,
            r: 0,
            q: 0,
            k: 0,
            p: 0,
        };
        const bishops = [];
        let numPieces = 0;
        let squareColor = 0;
        for (let i = Ox88.a8; i <= Ox88.h1; i++) {
            squareColor = (squareColor + 1) % 2;
            if (i & 136) {
                i += 7;
                continue;
            }
            const piece = this._board[i];
            if (piece) {
                pieces[piece.type] = piece.type in pieces ? pieces[piece.type] + 1 : 1;
                if (piece.type === BISHOP) {
                    bishops.push(squareColor);
                }
                numPieces++;
            }
        }
        if (numPieces === 2) {
            return true;
        } else if (
            // k vs. kn .... or .... k vs. kb
            numPieces === 3 &&
            (pieces[BISHOP] === 1 || pieces[KNIGHT] === 1)
        ) {
            return true;
        } else if (numPieces === pieces[BISHOP] + 2) {
            let sum = 0;
            const len = bishops.length;
            for (let i = 0; i < len; i++) {
                sum += bishops[i];
            }
            if (sum === 0 || sum === len) {
                return true;
```

```
843 |  |  |  |  |  |  |  |  }
844 |  |  |  |  |  |  |  }
845 |  |  |  |  |  |  |  return false;
846 |  |  |  |  |  |  }
847 |  |  |  |  |  |  isThreefoldRepetition() {
848 |  |  |  |  |  |  |  return this._getPositionCount(this._hash) >= 3;
849 |  |  |  |  |  |  }
850 |  |  |  |  |  |  isDrawByFiftyMoves() {
851 |  |  |  |  |  |  |  return this._halfMoves >= 100;
852 |  |  |  |  |  |  }
853 |  |  |  |  |  |  isDraw() {
854 |  |  |  |  |  |  |  return (
855 |  |  |  |  |  |  |  |  this.isDrawByFiftyMoves() ||
856 |  |  |  |  |  |  |  |  this.isStalemate() ||
857 |  |  |  |  |  |  |  |  this.isInsufficientMaterial() ||
858 |  |  |  |  |  |  |  |  this.isThreefoldRepetition()
859 |  |  |  |  |  |  |  );
860 |  |  |  |  |  |  }
861 |  |  |  |  |  |  isGameOver() {
862 |  |  |  |  |  |  |  return this.isCheckmate() || this.isDraw();
863 |  |  |  |  |  |  }
864 |  |  |  |  |  |  moves({ verbose = false, square = void 0, piece = void 0 } = {}) {
865 |  |  |  |  |  |  |  const moves = this._moves({ square, piece });
866 |  |  |  |  |  |  |  if (verbose) {
867 |  |  |  |  |  |  |  |  return moves.map((move) => new Move(this, move));
868 |  |  |  |  |  |  |  } else {
869 |  |  |  |  |  |  |  |  return moves.map((move) => this._moveToSan(move, moves));
870 |  |  |  |  |  |  |  }
871 |  |  |  |  |  |  }
872 |  |  |  |  |  |  _moves({ legal = true, piece = void 0, square = void 0 } = {}) {
873 |  |  |  |  |  |  |  const forSquare = square ? square.toLowerCase() : void 0;
874 |  |  |  |  |  |  |  const forPiece = piece?.toLowerCase();
875 |  |  |  |  |  |  |  const moves = [];
876 |  |  |  |  |  |  |  const us = this._turn;
877 |  |  |  |  |  |  |  const them = swapColor(us);
878 |  |  |  |  |  |  |  let firstSquare = 0x88.a8;
879 |  |  |  |  |  |  |  let lastSquare = 0x88.h1;
880 |  |  |  |  |  |  |  let singleSquare = false;
881 |  |  |  |  |  |  |  if (forSquare) {
882 |  |  |  |  |  |  |  |  if (!(forSquare in 0x88)) {
883 |  |  |  |  |  |  |  |  |  return [];
884 |  |  |  |  |  |  |  |  } else {
885 |  |  |  |  |  |  |  |  |  firstSquare = lastSquare = 0x88[forSquare];
886 |  |  |  |  |  |  |  |  |  singleSquare = true;
887 |  |  |  |  |  |  |  |  }
888 |  |  |  |  |  |  |  }
889 |  |  |  |  |  |  |  for (let from = firstSquare; from <= lastSquare; from++) {
890 |  |  |  |  |  |  |  |  if (from & 136) {
891 |  |  |  |  |  |  |  |  |  from += 7;
892 |  |  |  |  |  |  |  |  |  continue;
893 |  |  |  |  |  |  |  |  }
894 |  |  |  |  |  |  |  |  if (!this._board[from] || this._board[from].color === them) {
895 |  |  |  |  |  |  |  |  |  continue;
896 |  |  |  |  |  |  |  |  }
897 |  |  |  |  |  |  |  |  const { type } = this._board[from];
898 |  |  |  |  |  |  |  |  let to;
899 |  |  |  |  |  |  |  |  if (type === PAWN) {
900 |  |  |  |  |  |  |  |  |  if (forPiece && forPiece !== type) continue;
901 |  |  |  |  |  |  |  |  |  to = from + PAWN_OFFSETS[us][0];
902 |  |  |  |  |  |  |  |  |  if (!this._board[to]) {
903 |  |  |  |  |  |  |  |  |  |  addMove(moves, us, from, to, PAWN);
904 |  |  |  |  |  |  |  |  |  |  to = from + PAWN_OFFSETS[us][1];
905 |  |  |  |  |  |  |  |  |  |  if (SECOND_RANK[us] === rank(from) && !this._board[to]) {
906 |  |  |  |  |  |  |  |  |  |  |  addMove(moves, us, from, to, PAWN, void 0, BITS.BIG_PAWN);
907 |  |  |  |  |  |  |  |  |  |  }
908 |  |  |  |  |  |  |  |  |  }
909 |  |  |  |  |  |  |  |  |  for (let j = 2; j < 4; j++) {
910 |  |  |  |  |  |  |  |  |  |  to = from + PAWN_OFFSETS[us][j];
911 |  |  |  |  |  |  |  |  |  |  if (to & 136) continue;
912 |  |  |  |  |  |  |  |  |  |  if (this._board[to]?.color === them) {
913 |  |  |  |  |  |  |  |  |  |  |  addMove(moves, us, from, to, PAWN, this._board[to].type, BITS.CAPTURE);
```

```
        } else if (to === this._epSquare) {
          addMove(moves, us, from, to, PAWN, PAWN, BITS.EP_CAPTURE);
        }
      }
    } else {
      if (forPiece && forPiece !== type) continue;
      for (let j = 0, len = PIECE_OFFSETS[type].length; j < len; j++) {
        const offset = PIECE_OFFSETS[type][j];
        to = from;
        while (true) {
          to += offset;
          if (to & 136) break;
          if (!this._board[to]) {
            addMove(moves, us, from, to, type);
          } else {
            if (this._board[to].color === us) break;
            addMove(moves, us, from, to, type, this._board[to].type, BITS.CAPTURE);
            break;
          }
          if (type === KNIGHT || type === KING) break;
        }
      }
    }
    if (forPiece === void 0 || forPiece === KING) {
      if (!singleSquare || lastSquare === this._kings[us]) {
        if (this._castling[us] & BITS.KSIDE_CASTLE) {
          const castlingFrom = this._kings[us];
          const castlingTo = castlingFrom + 2;
          if (
            !this._board[castlingFrom + 1] &&
            !this._board[castlingTo] &&
            !this._attacked(them, this._kings[us]) &&
            !this._attacked(them, castlingFrom + 1) &&
            !this._attacked(them, castlingTo)
          ) {
            addMove(moves, us, this._kings[us], castlingTo, KING, void 0, BITS.KSIDE_CASTLE);
          }
        }
        if (this._castling[us] & BITS.QSIDE_CASTLE) {
          const castlingFrom = this._kings[us];
          const castlingTo = castlingFrom - 2;
          if (
            !this._board[castlingFrom - 1] &&
            !this._board[castlingFrom - 2] &&
            !this._board[castlingFrom - 3] &&
            !this._attacked(them, this._kings[us]) &&
            !this._attacked(them, castlingFrom - 1) &&
            !this._attacked(them, castlingTo)
          ) {
            addMove(moves, us, this._kings[us], castlingTo, KING, void 0, BITS.QSIDE_CASTLE);
          }
        }
      }
    }
    if (!legal || this._kings[us] === -1) {
      return moves;
    }
    const legalMoves = [];
    for (let i = 0, len = moves.length; i < len; i++) {
      this._makeMove(moves[i]);
      if (!this._isKingAttacked(us)) {
        legalMoves.push(moves[i]);
      }
      this._undoMove();
    }
    return legalMoves;
  }
  move(move) {
    let moveObj = null;
    const moves = this._moves();
```

```
        for (let i = 0, len = moves.length; i < len; i++) {
          if (
            move.from === algebraic(moves[i].from) &&
            move.to === algebraic(moves[i].to) &&
            (!('promotion' in moves[i]) || move.promotion === moves[i].promotion)
          ) {
            moveObj = moves[i];
            break;
          }
        }
        if (!moveObj) {
          throw new Error(`Invalid move: ${JSON.stringify(move)}`);
        }
        const prettyMove = new Move(this, moveObj);
        this._makeMove(moveObj);
        this._incPositionCount();
        return prettyMove;
      }
      _push(move) {
        this._history.push({
          move,
          kings: { b: this._kings.b, w: this._kings.w },
          turn: this._turn,
          castling: { b: this._castling.b, w: this._castling.w },
          epSquare: this._epSquare,
          halfMoves: this._halfMoves,
          moveNumber: this._moveNumber,
        });
      }
      _movePiece(from, to) {
        this._hash ^= this._pieceKey(from);
        this._board[to] = this._board[from];
        delete this._board[from];
        this._hash ^= this._pieceKey(to);
      }
      _makeMove(move) {
        const us = this._turn;
        const them = swapColor(us);
        this._push(move);
        if (move.flags & BITS.NULL_MOVE) {
          if (us === BLACK) {
            this._moveNumber++;
          }
          this._halfMoves++;
          this._turn = them;
          this._epSquare = EMPTY;
          return;
        }
        this._hash ^= this._epKey();
        this._hash ^= this._castlingKey();
        if (move.captured) {
          this._hash ^= this._pieceKey(move.to);
        }
        this._movePiece(move.from, move.to);
        if (move.flags & BITS.EP_CAPTURE) {
          if (this._turn === BLACK) {
            this._clear(move.to - 16);
          } else {
            this._clear(move.to + 16);
          }
        }
        if (move.promotion) {
          this._clear(move.to);
          this._set(move.to, { type: move.promotion, color: us });
        }
        if (this._board[move.to].type === KING) {
          this._kings[us] = move.to;
          if (move.flags & BITS.KSIDE_CASTLE) {
            const castlingTo = move.to - 1;
            const castlingFrom = move.to + 1;
            this._movePiece(castlingFrom, castlingTo);
```

```
        } else if (move.flags & BITS.QSIDE_CASTLE) {
          const castlingTo = move.to + 1;
          const castlingFrom = move.to - 2;
          this._movePiece(castlingFrom, castlingTo);
        }
        this._castling[us] = 0;
      }
      if (this._castling[us]) {
        for (let i = 0, len = ROOKS[us].length; i < len; i++) {
          if (move.from === ROOKS[us][i].square && this._castling[us] & ROOKS[us][i].flag) {
            this._castling[us] ^= ROOKS[us][i].flag;
            break;
          }
        }
      }
      if (this._castling[them]) {
        for (let i = 0, len = ROOKS[them].length; i < len; i++) {
          if (move.to === ROOKS[them][i].square && this._castling[them] & ROOKS[them][i].flag) {
            this._castling[them] ^= ROOKS[them][i].flag;
            break;
          }
        }
      }
      this._hash ^= this._castlingKey();
      if (move.flags & BITS.BIG_PAWN) {
        let epSquare;
        if (us === BLACK) {
          epSquare = move.to - 16;
        } else {
          epSquare = move.to + 16;
        }
        if (
          (!((move.to - 1) & 136) &&
            this._board[move.to - 1]?.type === PAWN &&
            this._board[move.to - 1]?.color === them) ||
          (!((move.to + 1) & 136) &&
            this._board[move.to + 1]?.type === PAWN &&
            this._board[move.to + 1]?.color === them)
        ) {
          this._epSquare = epSquare;
          this._hash ^= this._epKey();
        } else {
          this._epSquare = EMPTY;
        }
      } else {
        this._epSquare = EMPTY;
      }
      if (move.piece === PAWN) {
        this._halfMoves = 0;
      } else if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)) {
        this._halfMoves = 0;
      } else {
        this._halfMoves++;
      }
      if (us === BLACK) {
        this._moveNumber++;
      }
      this._turn = them;
      this._hash ^= SIDE_KEY;
    }
    undo() {
      const hash = this._hash;
      const move = this._undoMove();
      if (move) {
        const prettyMove = new Move(this, move);
        this._decPositionCount(hash);
        return prettyMove;
      }
      return null;
    }
    _undoMove() {
```

```
        const old = this._history.pop();
        if (old === void 0) {
            return null;
        }
        this._hash ^= this._epKey();
        this._hash ^= this._castlingKey();
        const move = old.move;
        this._kings = old.kings;
        this._turn = old.turn;
        this._castling = old.castling;
        this._epSquare = old.epSquare;
        this._halfMoves = old.halfMoves;
        this._moveNumber = old.moveNumber;
        this._hash ^= this._epKey();
        this._hash ^= this._castlingKey();
        this._hash ^= SIDE_KEY;
        const us = this._turn;
        const them = swapColor(us);
        if (move.flags & BITS.NULL_MOVE) {
            return move;
        }
        this._movePiece(move.to, move.from);
        if (move.piece) {
            this._clear(move.from);
            this._set(move.from, { type: move.piece, color: us });
        }
        if (move.captured) {
            if (move.flags & BITS.EP_CAPTURE) {
                let index;
                if (us === BLACK) {
                    index = move.to - 16;
                } else {
                    index = move.to + 16;
                }
                this._set(index, { type: PAWN, color: them });
            } else {
                this._set(move.to, { type: move.captured, color: them });
            }
        }
        if (move.flags & (BITS.KSIDE_CASTLE | BITS.QSIDE_CASTLE)) {
            let castlingTo, castlingFrom;
            if (move.flags & BITS.KSIDE_CASTLE) {
                castlingTo = move.to + 1;
                castlingFrom = move.to - 1;
            } else {
                castlingTo = move.to - 2;
                castlingFrom = move.to + 1;
            }
            this._movePiece(castlingFrom, castlingTo);
        }
        return move;
    }
    // Convert a move from 0×88 coordinates to Standard Algebraic Notation.
    _moveToSan(move, moves) {
        let output = '';
        if (move.flags & BITS.KSIDE_CASTLE) {
            output = 'O-O';
        } else if (move.flags & BITS.QSIDE_CASTLE) {
            output = 'O-O-O';
        } else {
            if (move.piece !== PAWN) {
                const disambiguator = getDisambiguator(move, moves);
                output += move.piece.toUpperCase() + disambiguator;
            }
            if (move.flags & (BITS.CAPTURE | BITS.EP_CAPTURE)) {
                if (move.piece === PAWN) {
                    output += algebraic(move.from)[0];
                }
                output += 'x';
            }
            output += algebraic(move.to);
```

```
        if (move.promotion) {
          output += '=' + move.promotion.toUpperCase();
        }
      }
      this._makeMove(move);
      if (this.isCheck()) {
        if (this.isCheckmate()) {
          output += '#';
        } else {
          output += '+';
        }
      }
      this._undoMove();
      return output;
    }
    turn() {
      return this._turn;
    }
    board() {
      const output = [];
      let row = [];
      for (let i = 0x88.a8; i <= 0x88.h1; i++) {
        if (this._board[i] == null) {
          row.push(null);
        } else {
          row.push({
            square: algebraic(i),
            type: this._board[i].type,
            color: this._board[i].color,
          });
        }
        if ((i + 1) & 136) {
          output.push(row);
          row = [];
          i += 8;
        }
      }
      return output;
    }
    squareColor(square) {
      if (square in 0x88) {
        const sq = 0x88[square];
        return (rank(sq) + file(sq)) % 2 === 0 ? 'light' : 'dark';
      }
      return null;
    }
    history({ verbose = false } = {}) {
      const reversedHistory = [];
      const moveHistory = [];
      while (this._history.length > 0) {
        reversedHistory.push(this._undoMove());
      }
      while (true) {
        const move = reversedHistory.pop();
        if (!move) {
          break;
        }
        if (verbose) {
          moveHistory.push(new Move(this, move));
        } else {
          moveHistory.push(this._moveToSan(move, this._moves()));
        }
        this._makeMove(move);
      }
      return moveHistory;
    }
    /*
     * Keeps track of position occurrence counts for the purpose of repetition
     * checking. Old positions are removed from the map if their counts are reduced to 0.
     */
    _getPositionCount(hash) {
```

```
        return this._positionCount.get(hash) ?? 0;
      }
      _incPositionCount() {
        this._positionCount.set(this._hash, (this._positionCount.get(this._hash) ?? 0) + 1);
      }
      _decPositionCount(hash) {
        const currentCount = this._positionCount.get(hash) ?? 0;
        if (currentCount === 1) {
          this._positionCount.delete(hash);
        } else {
          this._positionCount.set(hash, currentCount - 1);
        }
      }
      moveNumber() {
        return this._moveNumber;
      }
    }
    exports.BISHOP = BISHOP;
    exports.BLACK = BLACK;
    exports.Chess = Chess;
    exports.DEFAULT_POSITION = DEFAULT_POSITION;
    exports.KING = KING;
    exports.KNIGHT = KNIGHT;
    exports.Move = Move;
    exports.PAWN = PAWN;
    exports.QUEEN = QUEEN;
    exports.ROOK = ROOK;
    exports.SQUARES = SQUARES;
    exports.WHITE = WHITE;
    exports.validateFen = validateFen;
    exports.xoroshiro128 = xoroshiro128;
    Object.defineProperty(exports, Symbol.toStringTag, { value: 'Module' });
    return exports;
  })({});

  globalThis.TSChessShared = TSChessShared;
</script>

<script id="ai-worker-source-1" type="text/plain">
  !(function () {
    'use strict';
    const sharedCore = globalThis.TSChessShared;
    if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
    const Chess = sharedCore.Chess;
    sharedCore.SQUARES;
    const HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
      PIECE_VALUES = { p: 100, n: 320, b: 330, r: 500, q: 900, k: 0 },
      STARTING_NON_PAWN_MATERIAL =
        2 * (PIECE_VALUES.q + 2 * PIECE_VALUES.r + 2 * PIECE_VALUES.b + 2 * PIECE_VALUES.n),
      OPENING_PHASE_END_MATERIAL = STARTING_NON_PAWN_MATERIAL / 2;
    function searchRoot(chess, depth, ctx, previousBestMove) {
      const candidates = [],
        legalMoves = orderMoves(chess.moves({ verbose: !0 }), previousBestMove);
      for (const move of legalMoves) {
        (checkTime(ctx), chess.move(toMoveCommand(move)));
        const score = -negamax(chess, depth - 1, -1e7, 1e7, 1, ctx);
        (chess.undo(), candidates.push({ move: toMoveCommand(move), san: move.san, score: score }));
      }
      if (0 === candidates.length) throw new SearchTimeout();
      const principal = (function selectBestRootMove(candidates) {
          return candidates.reduce((currentBest, candidate) =>
            candidate.score > currentBest.score ? candidate : currentBest,
          );
        })(candidates),
        selection = (function selectRootMove(candidates, settings, best) {
          const level = settings.randomness / 30,
            windowCp = 20 + 150 * level,
            floorScore = best.score - windowCp,
            ticketPower = 4 - 1.5 * level;
          if (
            (function shouldForceBestMove(settings, best) {
```

```
        return (
          0 === settings.randomness ||
          (function isMateProtectedScore(score) {
            return Math.abs(score) >= 99e4;
          })(best.score) ||
          ('hard' === settings.preset && best.score < 0)
        );
      })(settings, best)
    )
      return {
        selected: best,
        debug: createDeterministicDebug(
          candidates,
          best,
          best,
          windowCp,
          floorScore,
          ticketPower,
        ),
      };
    const ticketCandidates = (function selectTicketCandidates(candidates, settings, best) {
      return 'hard' === settings.preset && best.score > 0
        ? candidates.filter((candidate) => candidate.score > 0)
        : candidates;
    })(candidates, settings, best),
    weightedCandidates = ticketCandidates
      .map((candidate) => ({
        candidate: candidate,
        tickets: (Math.max(0, candidate.score - floorScore) / windowCp) ** ticketPower,
      }))
      .filter(({ tickets: tickets }) => tickets > 0),
    totalTickets = weightedCandidates.reduce(
      (total, { tickets: tickets }) => total + tickets,
      0,
    );
    if (totalTickets <= 0)
      return {
        selected: best,
        debug: createDeterministicDebug(
          candidates,
          best,
          best,
          windowCp,
          floorScore,
          ticketPower,
        ),
      };
    let draw = Math.random() * totalTickets,
      selected = best;
    for (const { candidate: candidate, tickets: tickets } of weightedCandidates)
      if (((draw -= tickets), draw <= 0)) {
        selected = candidate;
        break;
      }
    return {
      selected: selected,
      debug: createWeightedDebug(
        candidates,
        best,
        selected,
        weightedCandidates,
        windowCp,
        floorScore,
        ticketPower,
        totalTickets,
      ),
    };
  })(candidates, ctx.settings, principal);
  return { selected: selection.selected, principal: principal, debug: selection.debug };
}
function shouldStopAfterStableDepth(result, previousPrincipal, settings) {
```

```
        return (
          'hard' === settings.preset &&
          null !== previousPrincipal &&
          !!isSameMoveCommand(previousPrincipal.move, result.principal.move) &&
          !(Math.abs(previousPrincipal.score - result.principal.score) > 90) &&
          (function rootScoreGap(debug) {
            const [bestCandidate, secondCandidate] = debug.candidates;
            return bestCandidate && secondCandidate ? bestCandidate.score - secondCandidate.score : 1e7;
          })(result.debug) >= 120
        );
      }
      function negamax(chess, depth, alpha, beta, ply, ctx) {
        if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
        if (depth <= 0)
          return ctx.settings.quiescence
            ? quiescence(chess, alpha, beta, ply, ctx)
            : evaluatePosition(chess);
        let bestScore = -1e7,
          currentAlpha = alpha;
        const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null);
        for (const move of legalMoves) {
          chess.move(toMoveCommand(move));
          const score = -negamax(chess, depth - 1, -beta, -currentAlpha, ply + 1, ctx);
          if (
            (chess.undo(),
            (bestScore = Math.max(bestScore, score)),
            (currentAlpha = Math.max(currentAlpha, score)),
            currentAlpha >= beta)
          )
            break;
        }
        return bestScore;
      }
      function quiescence(chess, alpha, beta, ply, ctx) {
        if ((checkTime(ctx), chess.isCheckmate() || chess.isDraw())) return terminalScore(chess, ply);
        let currentAlpha = alpha;
        const standPat = evaluatePosition(chess);
        if (standPat >= beta) return beta;
        if (((currentAlpha = Math.max(currentAlpha, standPat)), ply >= 6)) return currentAlpha;
        const legalMoves = orderMoves(chess.moves({ verbose: !0 }), null),
          tacticalMoves = chess.isCheck() ? legalMoves : legalMoves.filter(isTacticalMove);
        for (const move of tacticalMoves) {
          chess.move(toMoveCommand(move));
          const score = -quiescence(chess, -beta, -currentAlpha, ply + 1, ctx);
          if ((chess.undo(), score >= beta)) return beta;
          currentAlpha = Math.max(currentAlpha, score);
        }
        return currentAlpha;
      }
      function evaluatePosition(chess) {
        const board = chess.board(),
          pawns = [],
          bishops = { w: 0, b: 0 },
          kings = {};
        let whiteScore = 0,
          blackScore = 0,
          nonPawnMaterial = 0;
        for (const row of board)
          for (const pieceOnSquare of row) {
            if (null === pieceOnSquare) continue;
            const { color: color, type: type, square: square } = pieceOnSquare,
              pieceScore = PIECE_VALUES[type] + pieceSquareScore(type, color, square);
            ('w' === color ? (whiteScore += pieceScore) : (blackScore += pieceScore),
              'p' === type
                ? pawns.push({ color: color, file: fileIndex(square), rank: rankNumber(square) })
                : 'k' !== type && (nonPawnMaterial += PIECE_VALUES[type]),
              'b' === type && (bishops[color] += 1),
              'k' === type && (kings[color] = square));
          }
        ((whiteScore += bishops.w >= 2 ? 35 : 0),
          (blackScore += bishops.b >= 2 ? 35 : 0),
```

```
        (whiteScore += pawnStructureScore(pawns, 'w')),
        (blackScore += pawnStructureScore(pawns, 'b')),
        (whiteScore += openingPrinciplesScore(board, pawns, 'w', nonPawnMaterial)),
        (blackScore += openingPrinciplesScore(board, pawns, 'b', nonPawnMaterial)),
        (whiteScore += developmentScore(board, 'w')),
        (blackScore += developmentScore(board, 'b')),
        (whiteScore += kingSafetyScore(kings.w, pawns, 'w', nonPawnMaterial)),
        (blackScore += kingSafetyScore(kings.b, pawns, 'b', nonPawnMaterial));
      let score = (whiteScore - blackScore) * ('w' === chess.turn() ? 1 : -1);
      return ((score += 2 * chess.moves().length), chess.isCheck() && (score -= 45), score);
    }
    function pieceSquareScore(type, color, square) {
      const file = fileIndex(square),
        relativeRank = 'w' === color ? rankNumber(square) : 9 - rankNumber(square),
        centerDistance = Math.abs(file - 3.5) + Math.abs(rankNumber(square) - 4.5);
      switch (type) {
        case 'p':
          return 7 * relativeRank + (file >= 2 && file <= 5 ? 8 : 0);
        case 'n':
          return 42 - 12 * centerDistance - (1 === relativeRank ? 12 : 0);
        case 'b':
          return 30 - 7 * centerDistance;
        case 'r':
          return (relativeRank >= 7 ? 18 : 0) + (0 === file || 7 === file ? 4 : 0);
        case 'q':
          return 14 - 3 * centerDistance;
        case 'k':
          return relativeRank >= 7 ? -18 : 0;
      }
    }
    function pawnStructureScore(pawns, color) {
      const ownPawns = pawns.filter((pawn) => pawn.color === color),
        fileCounts = Array.from({ length: 8 }, () => 0);
      let score = 0;
      for (const pawn of ownPawns) fileCounts[pawn.file] += 1;
      for (const pawn of ownPawns) {
        const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
        (fileCounts[pawn.file] > 1 && (score -= 14),
          0 === (fileCounts[pawn.file - 1] ?? 0) &&
            0 === (fileCounts[pawn.file + 1] ?? 0) &&
            (score -= 10),
          isPassedPawn(pawn, pawns) && (score += (relativeRank - 1) * (relativeRank - 1) * 9));
      }
      return score;
    }
    function isPassedPawn(pawn, pawns) {
      const enemyColor = (function oppositeColor(color) {
        return 'w' === color ? 'b' : 'w';
      })(pawn.color);
      return !pawns.some(
        (candidate) =>
          !(candidate.color !== enemyColor || Math.abs(candidate.file - pawn.file) > 1) &&
          ('w' === pawn.color ? candidate.rank > pawn.rank : candidate.rank < pawn.rank),
      );
    }
    function openingPhaseWeight(nonPawnMaterial) {
      return clamp(
        (nonPawnMaterial - OPENING_PHASE_END_MATERIAL) /
          (STARTING_NON_PAWN_MATERIAL - OPENING_PHASE_END_MATERIAL),
        0,
        1,
      );
    }
    function openingPrinciplesScore(board, pawns, color, nonPawnMaterial) {
      const phase = openingPhaseWeight(nonPawnMaterial);
      if (phase <= 0) return 0;
      let score = (function centralLineOpeningScore(board, color) {
        let score = 0;
        return (
          hasPawnOnSquare(board, color, 'w' === color ? 'd2' : 'd7') || (score += 10),
          hasPawnOnSquare(board, color, 'w' === color ? 'e2' : 'e7') || (score += 12),
```

```
        score
      );
    })(board, color),
    developedCenterPawns = 0;
  for (const pawn of pawns) {
    if (pawn.color !== color) continue;
    const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
    3 === pawn.file || 4 === pawn.file
      ? (4 === relativeRank || 5 === relativeRank
        ? (score += 76)
        : 3 === relativeRank && (score += 30),
      relativeRank >= 3 && (developedCenterPawns += 1))
      : (2 !== pawn.file && 5 !== pawn.file) ||
        (4 === relativeRank || 5 === relativeRank
          ? (score += 18)
          : 3 === relativeRank && (score += 6));
  }
  return (developedCenterPawns >= 2 && (score += 16), Math.round(score * phase));
}
function hasPawnOnSquare(board, color, square) {
  for (const row of board)
    for (const piece of row)
      if (piece?.color === color && 'p' === piece.type && piece.square === square) return !0;
  return !1;
}
function developmentScore(board, color) {
  const homeRank = 'w' === color ? '1' : '8',
    knightHomeSquares = 'w' === color ? new Set(['b1', 'g1']) : new Set(['b8', 'g8']),
    bishopHomeSquares = 'w' === color ? new Set(['c1', 'f1']) : new Set(['c8', 'f8']);
  let score = 0;
  for (const row of board)
    for (const piece of row)
      null !== piece &&
        piece.color === color &&
        ('n' !== piece.type || knightHomeSquares.has(piece.square) || (score += 18),
        'b' !== piece.type || bishopHomeSquares.has(piece.square) || (score += 16),
        'q' === piece.type &&
          piece.square.endsWith(homeRank) &&
          hasMovedMinorPieces(board, color) &&
          (score -= 12));
  return score;
}
function hasMovedMinorPieces(board, color) {
  const homeSquares =
    'w' === color ? new Set(['b1', 'c1', 'f1', 'g1']) : new Set(['b8', 'c8', 'f8', 'g8']);
  for (const row of board)
    for (const piece of row)
      if (
        null !== piece &&
        piece.color === color &&
        ('n' === piece.type || 'b' === piece.type) &&
        homeSquares.has(piece.square)
      )
        return !1;
  return !0;
}
function kingSafetyScore(kingSquare, pawns, color, nonPawnMaterial) {
  if (!kingSquare || nonPawnMaterial < 1800) return 0;
  const kingFile = fileIndex(kingSquare),
    kingRank = rankNumber(kingSquare),
    shieldRank = 'w' === color ? kingRank + 1 : kingRank - 1;
  let score = 2 === kingFile || 6 === kingFile ? 28 : -18;
  for (let file = kingFile - 1; file <= kingFile + 1; file += 1)
    file < 0 ||
      file >= 8 ||
      (pawns.some(
        (pawn) => pawn.color === color && pawn.file === file && pawn.rank === shieldRank,
      )
        ? (score += 14)
        : (score +=
            openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) ?? -10));
```

```
        return score;
      }
      function openingCentralShieldScore(kingSquare, pawns, color, file, nonPawnMaterial) {
        const homeKingSquare = 'w' === color ? 'e1' : 'e8',
          phase = openingPhaseWeight(nonPawnMaterial);
        return kingSquare !== homeKingSquare || phase <= 0 || (3 !== file && 4 !== file)
          ? null
          : pawns.some((pawn) => {
              if (pawn.color !== color || pawn.file !== file) return !1;
              const relativeRank = 'w' === color ? pawn.rank : 9 - pawn.rank;
              return 3 === relativeRank || 4 === relativeRank;
            })
            ? Math.round(24 * phase - 10)
            : null;
      }
      function terminalScore(chess, ply) {
        return chess.isCheckmate() ? -1e6 + ply : chess.isDraw() ? 0 : evaluatePosition(chess);
      }
      function orderMoves(moves, preferredMove) {
        return [ ... moves].sort(
          (left, right) =>
            moveOrderingScore(right, preferredMove) - moveOrderingScore(left, preferredMove),
        );
      }
      function moveOrderingScore(move, preferredMove) {
        let score = 0;
        return (
          preferredMove &&
          (function isSameMove(move, command) {
            return (
              move.from === command.from &&
              move.to === command.to &&
              (move.promotion ?? void 0) === (command.promotion ?? void 0)
            );
          })(move, preferredMove) &&
          (score += 2e6),
          move.san.includes('#') ? (score += 15e5) : move.san.includes('+') && (score += 16e3),
          move.isCapture() &&
            (score +=
              12 * (move.captured ? PIECE_VALUES[move.captured] : PIECE_VALUES.p) -
              PIECE_VALUES[move.piece]),
          move.isPromotion() && move.promotion && (score += PIECE_VALUES[move.promotion] + 8e3),
          (move.isKingsideCastle() || move.isQueensideCastle()) && (score += 1500),
          (score += 2 * pieceSquareScore(move.piece, move.color, move.to)),
          score
        );
      }
      function checkTime(ctx) {
        if (((ctx.nodes += 1), ctx.nodes % 2048 !== 0)) return;
        const now = Date.now();
        if (
          (now - ctx.lastProgressAt >= 100 &&
            publishSearchProgress(ctx, ctx.progressState, now - ctx.startTime, now),
          now >= ctx.deadline)
        )
          throw new SearchTimeout();
      }
      function publishSearchProgress(
        ctx,
        progressState,
        elapsedMs = Date.now() - ctx.startTime,
        progressAt = Date.now(),
      ) {
        ((ctx.progressState = progressState),
          (ctx.lastProgressAt = progressAt),
          ctx.onProgress?.({
            kind: 'progress',
            id: ctx.requestId,
            move: progressState.move,
            depthReached: progressState.depthReached,
            nodes: ctx.nodes,
```

```
        elapsedMs: elapsedMs,
        score: progressState.score,
        debug: progressState.debug,
      }));
    }
    function isTacticalMove(move) {
      return (
        move.isCapture() || move.isPromotion() || move.san.includes('+') || move.san.includes('#')
      );
    }
    function createDeterministicDebug(
      candidates,
      principal,
      selected,
      windowCp,
      floorScore,
      ticketPower,
    ) {
      return {
        candidates: sortCandidatesByScore(candidates).map((candidate) => ({
          move: candidate.move,
          san: candidate.san,
          score: candidate.score,
          tickets: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
          probability: isSameMoveCommand(candidate.move, selected.move) ? 1 : 0,
          selected: isSameMoveCommand(candidate.move, selected.move),
          principal: isSameMoveCommand(candidate.move, principal.move),
        })),
        bestScore: principal.score,
        windowCp: windowCp,
        floorScore: floorScore,
        ticketPower: ticketPower,
        totalTickets: 1,
      };
    }
    function createWeightedDebug(
      candidates,
      principal,
      selected,
      weightedCandidates,
      windowCp,
      floorScore,
      ticketPower,
      totalTickets,
    ) {
      return {
        candidates: sortCandidatesByScore(candidates).map((candidate) => {
          const weightedCandidate = weightedCandidates.find(({ candidate: weighted }) =>
            isSameMoveCommand(weighted.move, candidate.move),
          ),
          tickets = weightedCandidate?.tickets ?? 0;
          return {
            move: candidate.move,
            san: candidate.san,
            score: candidate.score,
            tickets: tickets,
            probability: totalTickets > 0 ? tickets / totalTickets : 0,
            selected: isSameMoveCommand(candidate.move, selected.move),
            principal: isSameMoveCommand(candidate.move, principal.move),
          };
        }),
        bestScore: principal.score,
        windowCp: windowCp,
        floorScore: floorScore,
        ticketPower: ticketPower,
        totalTickets: totalTickets,
      };
    }
    function sortCandidatesByScore(candidates) {
      return [...candidates].sort((left, right) => right.score - left.score);
    }
```

```
function toMoveCommand(move) {
  return move.promotion
    ? { from: move.from, to: move.to, promotion: move.promotion }
    : { from: move.from, to: move.to };
}
function isSameMoveCommand(left, right) {
  return (
    left.from === right.from &&
    left.to === right.to &&
    (left.promotion ?? void 0) === (right.promotion ?? void 0)
  );
}
function fileIndex(square) {
  return square.charCodeAt(0) - 97;
}
function rankNumber(square) {
  return Number(square[1]);
}
function clamp(value, min, max) {
  return Math.min(max, Math.max(min, value));
}
class SearchTimeout extends Error {}
const workerScope = globalThis;
workerScope.addEventListener('message', (event) => {
  const response = (function searchBestMove(request, onProgress) {
    const chess = new Chess(request.fen),
      settings = (function normalizeAiSettings(settings) {
        const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
        return {
          preset: settings.preset,
          maxDepth:
            ((depth = settings.maxDepth),
            (preset = settings.preset),
            'hard' === preset ? HARD_MAX_DEPTH : clamp(Math.round(depth), 1, 5)),
          timeLimitMs: timeLimitMs,
          randomness: 30,
          quiescence: !0,
        };
        var depth, preset;
      })(request.settings),
      legalMoves = chess.moves({ verbose: !0 }),
      startTime = Date.now(),
      ctx = {
        requestId: request.id,
        settings: settings,
        deadline: startTime + settings.timeLimitMs,
        startTime: startTime,
        nodes: 0,
        onProgress: onProgress,
        lastProgressAt: startTime,
        progressState: { move: null, depthReached: 0, score: 0 },
      };
    if (0 === legalMoves.length)
      return {
        id: request.id,
        move: null,
        depthReached: 0,
        nodes: 0,
        elapsedMs: 0,
        score: terminalScore(chess, 0),
      };
    let debug,
      best = null,
      previousBestMove = null,
      previousPrincipal = null,
      depthReached = 0;
    for (let depth = 1; depth <= settings.maxDepth; depth += 1) {
      try {
        const result = searchRoot(chess, depth, ctx, previousBestMove),
          elapsedMs = Date.now() - startTime;
        if (
```

```
                ((best = result.selected),
                (debug = result.debug),
                (previousBestMove = result.principal.move),
                (depthReached = depth),
                publishSearchProgress(
                  ctx,
                  { move: best.move, depthReached: depthReached, score: best.score, debug: debug },
                  elapsedMs,
                ),
                shouldStopAfterStableDepth(result, previousPrincipal, settings))
              )
              break;
            previousPrincipal = result.principal;
          } catch (error) {
            if (error instanceof SearchTimeout) break;
            throw error;
          }
          if (Date.now() >= ctx.deadline) break;
        }
        return (
          (best ??= (function chooseFallbackMove(chess) {
            const [move] = orderMoves(chess.moves({ verbose: !0 }), null);
            if (!move) return { move: { from: 'a1', to: 'a1' }, score: terminalScore(chess, 0) };
            chess.move(toMoveCommand(move));
            const score = -evaluatePosition(chess);
            return (chess.undo(), { move: toMoveCommand(move), score: score });
          })(chess)),
          {
            id: request.id,
            move: best.move,
            depthReached: depthReached,
            nodes: ctx.nodes,
            elapsedMs: Date.now() - startTime,
            score: best.score,
            debug: debug,
          }
        );
      })(event.data, (progress) => {
        workerScope.postMessage(progress);
      });
      workerScope.postMessage({ ...response, kind: 'result' });
    });
  })();
</script>

<script>
  (() => {
    const element = document.getElementById('ts-chess-shared-core-source');
    if (!element) {
      throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
    }
    new Function(element.textContent ?? '')();
  })();
</script>
<script src="./main.js" defer></script>
<link rel="stylesheet" href="./style.css" />
</head>
<body>
  <main id="chess-app" class="chess-app">
    <aside class="control-panel" aria-label="게임 컨트롤">
      <section class="control-group" aria-label="모드">
        <span class="control-label">모드</span>
        <div class="segmented-control" role="group" aria-label="플레이 모드">
          <button class="segmented-button" type="button" data-mode="local">로컬 2인</button>
          <button class="segmented-button" type="button" data-mode="ai">AI 상대</button>
        </div>
      </section>

      <section id="color-controls" class="control-group" aria-label="진영">
        <span class="control-label">진영</span>
        <div class="segmented-control" role="group" aria-label="인간 진영">
```

```
        <button class="segmented-button" type="button" data-color="w">백</button>
        <button class="segmented-button" type="button" data-color="b">흑</button>
      </div>
    </section>

    <section id="ai-controls" class="control-group ai-settings" aria-label="AI 설정">
      <span class="control-label">AI 설정</span>
      <div
        class="segmented-control segmented-control-three"
        role="group"
        aria-label="AI 프리셋"
      >
        <button class="segmented-button" type="button" data-preset="easy">쉬움</button>
        <button class="segmented-button" type="button" data-preset="normal">보통</button>
        <button class="segmented-button" type="button" data-preset="hard">어려움</button>
      </div>
      <label class="range-control">
        <span>최대 깊이 <strong id="ai-depth-value"></strong></span>
        <input id="ai-depth-input" type="range" min="1" max="5" step="1" />
      </label>
      <label class="range-control">
        <span>시간 제한 <strong id="ai-time-value"></strong></span>
        <input id="ai-time-input" type="range" min="500" max="5000" step="250" />
      </label>
    </section>

    <section class="control-group clock-settings" aria-label="시간 설정">
      <span class="control-label">시간</span>
      <div class="clock-preset-control" role="group" aria-label="시간 프리셋">
        <button class="segmented-button" type="button" data-clock-preset="1/1">1/1</button>
        <button class="segmented-button" type="button" data-clock-preset="3/2">3/2</button>
        <button class="segmented-button" type="button" data-clock-preset="5/3">5/3</button>
        <button class="segmented-button" type="button" data-clock-preset="10/5">10/5</button>
        <button class="segmented-button" type="button" data-clock-preset="15/10">15/10</button>
      </div>
    </section>

    <section class="action-row" aria-label="게임 동작">
      <button id="new-game-button" class="primary-button" type="button">새 게임</button>
      <button id="undo-button" class="ghost-button" type="button">되돌리기</button>
    </section>
  </aside>

  <section class="board-stage" aria-label="체스판">
    <div id="chess-board" class="chess-board" role="grid" aria-label="체스판"></div>
  </section>

  <aside class="clock-panel" aria-label="체스 타이머">
    <div id="clock-display" class="clock-display">
      <div id="white-clock" class="clock-card">
        <span class="clock-side">백</span>
        <strong id="white-clock-time" class="clock-time"></strong>
      </div>
      <div id="black-clock" class="clock-card">
        <span class="clock-side">흑</span>
        <strong id="black-clock-time" class="clock-time"></strong>
      </div>
    </div>
  </aside>

  <aside class="info-panel" aria-label="게임 정보">
    <section class="status-panel" aria-live="polite">
      <span class="control-label">상태</span>
      <strong id="game-status" class="game-status"></strong>
      <dl class="status-grid">
        <div>
          <dt>차례</dt>
          <dd id="turn-meta"></dd>
        </div>
        <div>
          <dt>모드</dt>
```

```
                        <dd id="mode-meta"></dd>
                    </div>
                    <div>
                        <dt>수순</dt>
                        <dd id="move-meta"></dd>
                    </div>
                </dl>
            </section>

            <section id="ai-candidate-panel" class="ai-candidate-panel" aria-label="AI 정보" hidden>
                <div class="ai-candidate-header">
                    <span class="control-label">AI 정보</span>
                    <span id="ai-candidate-meta" class="ai-candidate-meta"></span>
                </div>
                <dl class="ai-info-grid">
                    <div>
                        <dt>상태</dt>
                        <dd id="ai-info-status"></dd>
                    </div>
                    <div>
                        <dt>난이도</dt>
                        <dd id="ai-info-preset"></dd>
                    </div>
                    <div>
                        <dt>인간</dt>
                        <dd id="ai-info-human"></dd>
                    </div>
                    <div>
                        <dt>depth</dt>
                        <dd id="ai-info-depth"></dd>
                    </div>
                    <div>
                        <dt>nodes</dt>
                        <dd id="ai-info-nodes"></dd>
                    </div>
                    <div>
                        <dt>score</dt>
                        <dd id="ai-info-score"></dd>
                    </div>
                </dl>
                <div class="ai-candidate-table" role="table" aria-label="AI 후보 수 평가">
                    <div class="ai-candidate-row ai-candidate-head" role="row">
                        <span role="columnheader">SAN</span>
                        <span role="columnheader">cp</span>
                        <span role="columnheader">weight</span>
                        <span role="columnheader">prob</span>
                    </div>
                    <div id="ai-candidate-list" class="ai-candidate-list"></div>
                </div>
            </section>

            <section class="history-panel" aria-label="기보">
                <div class="history-header">
                    <span class="control-label">기보</span>
                </div>
                <div id="move-list" class="move-list"></div>
            </section>
        </aside>

        <div id="promotion-overlay" class="promotion-overlay" hidden>
            <div
                class="promotion-dialog"
                role="dialog"
                aria-modal="true"
                aria-labelledby="promotion-title"
            >
                <h2 id="promotion-title">승격</h2>
                <div
                    id="promotion-choices"
                    class="promotion-choices"
                    role="group"
```

```
2050 ¦ ····aria-label="승격 기물"
2051 ¦ ····></div>
2052 ¦ ··</div>
2053 ¦ ··</div>
2054 ¦ ··</main>
2055 ¦ ··</body>
2056 ¦ </html>
2057 ¦
```

==== main.js (33823 B / 04a8e344941e) ====

```
  1 ¦ const sharedCore = globalThis.TSChessShared;
  2 ¦ if (!sharedCore) throw new Error('Missing shared chess runtime: TSChessShared');
  3 ¦ const Chess = sharedCore.Chess,
  4 ¦   SQUARES = sharedCore.SQUARES,
  5 ¦   HARD_MAX_DEPTH = Number.POSITIVE_INFINITY,
  6 ¦   AI_PRESETS = {
  7 ¦     easy: { preset: 'easy', maxDepth: 2, timeLimitMs: 750, randomness: 30, quiescence: !0 },
  8 ¦     normal: { preset: 'normal', maxDepth: 5, timeLimitMs: 5e3, randomness: 30, quiescence: !0 },
  9 ¦     hard: {
 10 ¦       preset: 'hard',
 11 ¦       maxDepth: HARD_MAX_DEPTH,
 12 ¦       timeLimitMs: 5e3,
 13 ¦       randomness: 30,
 14 ¦       quiescence: !0,
 15 ¦     },
 16 ¦   },
 17 ¦   DEFAULT_AI_SETTINGS = { ...AI_PRESETS.normal };
 18 ¦ function normalizeAiSettings(settings) {
 19 ¦   const timeLimitMs = Math.max(100, Math.round(settings.timeLimitMs));
 20 ¦   return {
 21 ¦     preset: settings.preset,
 22 ¦     maxDepth:
 23 ¦       ((depth = settings.maxDepth),
 24 ¦       (preset = settings.preset),
 25 ¦       'hard' === preset
 26 ¦         ? HARD_MAX_DEPTH
 27 ¦         : (function clamp(value, min, max) {
 28 ¦             return Math.min(max, Math.max(min, value));
 29 ¦           })(Math.round(depth), 1, 5)),
 30 ¦     timeLimitMs: timeLimitMs,
 31 ¦     randomness: 30,
 32 ¦     quiescence: !0,
 33 ¦   };
 34 ¦   var depth, preset;
 35 ¦ }
 36 ¦ const jsContent = (() => {
 37 ¦     const sharedElement = document.getElementById('ts-chess-shared-core-source');
 38 ¦     const workerElement = document.getElementById('ai-worker-source-1');
 39 ¦     if (!sharedElement) {
 40 ¦       throw new Error('Missing shared chess runtime source: ts-chess-shared-core-source');
 41 ¦     }
 42 ¦     if (!workerElement) {
 43 ¦       throw new Error('Missing inline worker source: ai-worker-source-1');
 44 ¦     }
 45 ¦     return (sharedElement.textContent ?? '') + '\n' + (workerElement.textContent ?? '');
 46 ¦   })(),
 47 ¦   blob =
 48 ¦     'undefined' != typeof self &&
 49 ¦     self.Blob &&
 50 ¦     new Blob(['(self.URL ?? self.webkitURL).revokeObjectURL(self.location.href);', jsContent], {
 51 ¦       type: 'text/javascript;charset=utf-8',
 52 ¦     });
 53 ¦ function WorkerWrapper(options) {
 54 ¦   let objURL;
 55 ¦   try {
 56 ¦     if (((objURL = blob && (self.URL ?? self.webkitURL).createObjectURL(blob)), !objURL)) throw '';
 57 ¦     const worker = new Worker(objURL, { name: options?.name });
 58 ¦     return (
 59 ¦       worker.addEventListener('error', () => {
 60 ¦         (self.URL ?? self.webkitURL).revokeObjectURL(objURL);
 61 ¦       }),
```

```
        worker
      );
    } catch (e) {
      return new Worker('data:text/javascript;charset=utf-8,' + encodeURIComponent(jsContent), {
        name: options?.name,
      });
    }
  }
  const BOARD_FILES = ['a', 'b', 'c', 'd', 'e', 'f', 'g', 'h'],
    BLACK_FILES = [...BOARD_FILES].reverse(),
    WHITE_RANKS = [8, 7, 6, 5, 4, 3, 2, 1],
    BLACK_RANKS = [...WHITE_RANKS].reverse(),
    PROMOTION_PIECES = ['q', 'r', 'b', 'n'],
    SQUARE_SET = new Set(SQUARES),
    AI_WHITE_OPENING_PRESET = [
      { from: 'e2', to: 'e4', weight: 28 },
      { from: 'd2', to: 'd4', weight: 25 },
      { from: 'g1', to: 'f3', weight: 24 },
      { from: 'c2', to: 'c4', weight: 20 },
      { from: 'g2', to: 'g3', weight: 9 },
      { from: 'b2', to: 'b3', weight: 6 },
    ],
    CLOCK_PRESETS = {
      '1/1': { baseMs: 6e4, incrementMs: 1e3 },
      '3/2': { baseMs: 18e4, incrementMs: 2e3 },
      '5/3': { baseMs: 3e5, incrementMs: 3e3 },
      '10/5': { baseMs: 6e5, incrementMs: 5e3 },
      '15/10': { baseMs: 9e5, incrementMs: 1e4 },
    },
    PIECE_GLYPHS = {
      w: { p: '♙', n: '♘', b: '♗', r: '♖', q: '♕', k: '♔' },
      b: { p: '♟', n: '♞', b: '♝', r: '♜', q: '♛', k: '♚' },
    };
  function requireElement(selector, root) {
    const element = root.querySelector(selector);
    if (!element) throw new Error(`Missing element: ${selector}`);
    return element;
  }
  function createCoordinate(type, label) {
    const coordinate = document.createElement('span');
    return (
      (coordinate.className = `coordinate coordinate-${type}`),
      (coordinate.textContent = label),
      coordinate
    );
  }
  function toSquare(file, rank) {
    return `${file}${rank}`;
  }
  function createClockState(preset) {
    const baseMs = CLOCK_PRESETS[preset].baseMs;
    return { w: baseMs, b: baseMs };
  }
  function colorName(color) {
    return 'w' === color ? '백' : '흑';
  }
  function presetName(preset) {
    switch (preset) {
      case 'easy':
        return '쉬움';
      case 'normal':
        return '보통';
      case 'hard':
        return '어려움';
    }
  }
  function formatSeconds(milliseconds) {
    return `${(milliseconds / 1e3).toFixed(milliseconds % 1e3 == 0 ? 0 : 2)}초`;
  }
  function formatDebugScore(score) {
    return Math.abs(score) >= 9e5
```

```
        ? score > 0
            ? '+mate'
            : '-mate'
        : score > 0
            ? `+${Math.round(score)}`
            : String(Math.round(score));
}
function oppositeColor(color) {
    return 'w' === color ? 'b' : 'w';
}
function getSquareLabel(square, piece) {
    return piece ? `${square} ${colorName(piece.color)} ${pieceName(piece.type)}` : `${square} 빈 칸`;
}
function pieceName(piece) {
    switch (piece) {
        case 'p':
            return '폰';
        case 'n':
            return '나이트';
        case 'b':
            return '비숍';
        case 'r':
            return '룩';
        case 'q':
            return '퀸';
        case 'k':
            return '킹';
    }
}
!(function createChessApp() {
    const root = requireElement('#chess-app', document),
        boardElement = requireElement('#chess-board', root),
        colorControlsElement = requireElement('#color-controls', root),
        aiControlsElement = requireElement('#ai-controls', root),
        statusElement = requireElement('#game-status', root),
        turnMetaElement = requireElement('#turn-meta', root),
        modeMetaElement = requireElement('#mode-meta', root),
        moveMetaElement = requireElement('#move-meta', root),
        aiCandidatePanelElement = requireElement('#ai-candidate-panel', root),
        aiCandidateMetaElement = requireElement('#ai-candidate-meta', root),
        aiInfoStatusElement = requireElement('#ai-info-status', root),
        aiInfoPresetElement = requireElement('#ai-info-preset', root),
        aiInfoHumanElement = requireElement('#ai-info-human', root),
        aiInfoDepthElement = requireElement('#ai-info-depth', root),
        aiInfoNodesElement = requireElement('#ai-info-nodes', root),
        aiInfoScoreElement = requireElement('#ai-info-score', root),
        aiCandidateListElement = requireElement('#ai-candidate-list', root),
        moveListElement = requireElement('#move-list', root),
        clockDisplayElement = requireElement('#clock-display', root),
        whiteClockElement = requireElement('#white-clock', root),
        blackClockElement = requireElement('#black-clock', root),
        whiteClockTimeElement = requireElement('#white-clock-time', root),
        blackClockTimeElement = requireElement('#black-clock-time', root),
        newGameButton = requireElement('#new-game-button', root),
        undoButton = requireElement('#undo-button', root),
        aiDepthInput = requireElement('#ai-depth-input', root),
        aiDepthValue = requireElement('#ai-depth-value', root),
        aiTimeInput = requireElement('#ai-time-input', root),
        aiTimeValue = requireElement('#ai-time-value', root),
        promotionOverlay = requireElement('#promotion-overlay', root),
        promotionChoices = requireElement('#promotion-choices', root),
        modeButtons = Array.from(root.querySelectorAll('[data-mode]')),
        colorButtons = Array.from(root.querySelectorAll('[data-color]')),
        presetButtons = Array.from(root.querySelectorAll('[data-preset]')),
        clockPresetButtons = Array.from(root.querySelectorAll('[data-clock-preset]')),
        game = new Chess();
    let mode = 'local',
        clockPreset = '10/5',
        clockMs = createClockState(clockPreset),
        activeClockColor = null,
        lastClockTick = Date.now(),
```

```
    clockTimerId = null,
    timedOutColor = null,
    humanColor = 'w',
    selectedSquare = null,
    legalMoves = [],
    pendingPromotion = null,
    aiThinking = !1,
    aiTimerId = null,
    aiWorker = null,
    aiRequestId = 0,
    activeAiRequestId = null,
    activeAiFen = null,
    aiSettings = { ...DEFAULT_AI_SETTINGS },
    aiProgress = null;
  function selectSquare(square) {
    ((selectedSquare = square),
      (legalMoves = game.moves({ verbose: !0, square: square })),
      render());
  }
  function playMove(moveCommand) {
    if (!commitMove(moveCommand)) return (clearSelection(), void render());
    ((pendingPromotion = null), clearSelection(), render(), scheduleAiIfNeeded());
  }
  function resetGame() {
    (cancelAiMove(),
      game.reset(),
      (pendingPromotion = null),
      clearSelection(),
      resetClockState(),
      restartClockForCurrentTurn(),
      render(),
      scheduleAiIfNeeded());
  }
  function scheduleAiIfNeeded() {
    'ai' !== mode ||
      game.isGameOver() ||
      null !== timedOutColor ||
      game.turn() === humanColor ||
      aiThinking ||
      pendingPromotion ||
      (function tryCommitAiWhiteOpeningPreset() {
        if ('b' !== humanColor || 'w' !== game.turn() || game.history().length > 0) return !1;
        const selectedMove = (function pickWeightedOpeningMove(moves) {
          const totalWeight = moves.reduce((total, move) => total + move.weight, 0);
          if (totalWeight <= 0) return null;
          let draw = Math.random() * totalWeight;
          for (const move of moves)
            if (((draw -= move.weight), draw <= 0))
              return { from: move.from, to: move.to, promotion: move.promotion };
          const fallback = moves.at(-1);
          return fallback
            ? { from: fallback.from, to: fallback.to, promotion: fallback.promotion }
            : null;
        })(
          AI_WHITE_OPENING_PRESET.filter((presetMove) =>
            game.moves({ verbose: !0, square: presetMove.from }).some((move) =>
              (function isSameMoveCommand(move, command) {
                return (
                  move.from === command.from &&
                  move.to === command.to &&
                  (move.promotion ?? void 0) === (command.promotion ?? void 0)
                );
              })(move, presetMove),
            ),
          ),
        );
        return !(
          !selectedMove ||
          !commitMove(selectedMove) ||
          ((aiProgress = null), clearSelection(), render(), 0)
        );
```

```
})()
((aiThinking = !0),
(aiProgress = null),
render(),
(aiTimerId = window.setTimeout(() => {
((aiTimerId = null),
(function startAiSearch() {
aiWorker?.terminate();
const requestId = aiRequestId + 1,
fen = game.fen();
((aiRequestId = requestId),
(activeAiRequestId = requestId),
(activeAiFen = fen),
(aiWorker = new WorkerWrapper()),
aiWorker.addEventListener('message', (event) => {
!(function handleAiMessage(message) {
if (
message.id === activeAiRequestId &&
null !== activeAiFen &&
game.fen() === activeAiFen
) {
if (((aiProgress = message), 'progress' === message.kind))
return (renderStatus(), void renderAiCandidateDebug());
((aiThinking = !1),
(activeAiRequestId = null),
(activeAiFen = null),
aiWorker?.terminate(),
(aiWorker = null),
message.move && commitMove(message.move),
clearSelection(),
render());
}
})(event.data);
}),
aiWorker.addEventListener('error', () => {
activeAiRequestId === requestId &&
((aiThinking = !1),
(activeAiRequestId = null),
(activeAiFen = null),
aiWorker?.terminate(),
(aiWorker = null),
render());
}),
aiWorker.postMessage({
id: requestId,
fen: fen,
settings: resolveAiSettingsForSearch(),
}));
})());
}, 180)));
}
function cancelAiMove() {
((aiRequestId += 1),
(activeAiRequestId = null),
(activeAiFen = null),
(aiProgress = null),
null !== aiTimerId && (window.clearTimeout(aiTimerId), (aiTimerId = null)),
aiWorker && (aiWorker.terminate(), (aiWorker = null)),
(aiThinking = !1));
}
function commitMove(moveCommand) {
const movingColor = game.turn();
if (!flushClockElapsed()) return !1;
try {
game.move(moveCommand);
} catch {
return !1;
}
return (
(clockMs[movingColor] += CLOCK_PRESETS[clockPreset].incrementMs),
restartClockForCurrentTurn(),
```

```
!0
);
}
function updateAiSettings(settings) {
((aiSettings = normalizeAiSettings(settings)),
aiThinking && cancelAiMove(),
render(),
scheduleAiIfNeeded());
}
function resolveAiSettingsForSearch() {
const settings = normalizeAiSettings(aiSettings);
if ('hard' !== settings.preset) return settings;
const aiColor = game.turn(),
remainingMs = clockMs[aiColor],
targetMs = remainingMs / 30 + 0.8 * CLOCK_PRESETS[clockPreset].incrementMs,
timeLimitMs = remainingMs <= 3e3 ? Math.min(targetMs, 0.35 * remainingMs) : targetMs;
return normalizeAiSettings({ ...settings, timeLimitMs: timeLimitMs });
}
function resetClockState() {
((clockMs = createClockState(clockPreset)),
(timedOutColor = null),
(activeClockColor = null),
(lastClockTick = Date.now()));
}
function restartClockForCurrentTurn() {
if ((clearClockTimer(), game.isGameOver() || null !== timedOutColor))
return ((activeClockColor = null), void renderClocks());
((activeClockColor = game.turn()),
(lastClockTick = Date.now()),
(clockTimerId = window.setInterval(tickClock, 100)),
renderClocks());
}
function clearClockTimer() {
null !== clockTimerId && (window.clearInterval(clockTimerId), (clockTimerId = null));
}
function tickClock() {
flushClockElapsed() && renderClocks();
}
function flushClockElapsed() {
if (null === activeClockColor || null !== timedOutColor || game.isGameOver()) return !0;
const now = Date.now(),
elapsedMs = Math.max(0, now - lastClockTick);
return (
(lastClockTick = now),
0 === elapsedMs ||
((clockMs[activeClockColor] = Math.max(0, clockMs[activeClockColor] - elapsedMs)),
0 !== clockMs[activeClockColor] ||
((function handleClockTimeout(color) {
((timedOutColor = color),
(activeClockColor = null),
(clockMs[color] = 0),
clearClockTimer(),
cancelAiMove(),
clearSelection(),
render());
})(activeClockColor),
!1))
);
}
function clearSelection() {
((selectedSquare = null), (legalMoves = []));
}
function render() {
(!(function renderControls() {
(modeButtons.forEach((button) => {
const isActive = button.dataset.mode === mode;
(button.classList.toggle('is-active', isActive),
button.setAttribute('aria-pressed', String(isActive)));
}),
colorButtons.forEach((button) => {
const isActive = button.dataset.color === humanColor;
```

```
        (button.classList.toggle('is-active', isActive),
          button.setAttribute('aria-pressed', String(isActive)));
      }),
      (colorControlsElement.hidden = 'ai' !== mode),
      (aiControlsElement.hidden = 'ai' !== mode),
      (function renderAiSettingsControls() {
        (presetButtons.forEach((button) => {
          const isActive = button.dataset.preset === aiSettings.preset;
          (button.classList.toggle('is-active', isActive),
            button.setAttribute('aria-pressed', String(isActive)));
        }),
          (aiDepthInput.value =
            'hard' === aiSettings.preset ? aiDepthInput.max : String(aiSettings.maxDepth)),
          (aiDepthInput.disabled = 'hard' === aiSettings.preset),
          (aiDepthValue.textContent =
            'hard' === aiSettings.preset ? '무제한' : `${aiSettings.maxDepth}`),
          (aiTimeInput.value = String(aiSettings.timeLimitMs)),
          (aiTimeInput.disabled = 'hard' === aiSettings.preset),
          (aiTimeValue.textContent =
            'hard' === aiSettings.preset ? '자동' : `${formatSeconds(aiSettings.timeLimitMs)}`));
      })(),
      (function renderClockSettingsControls() {
        clockPresetButtons.forEach((button) => {
          const isActive = button.dataset.clockPreset === clockPreset;
          (button.classList.toggle('is-active', isActive),
            button.setAttribute('aria-pressed', String(isActive)));
        });
      })(),
      (undoButton.disabled = 0 === game.history().length));
  })(),
    renderClocks(),
    (function renderBoard() {
      const fragment = document.createDocumentFragment(),
        orientation = getBoardOrientation(),
        files = 'w' === orientation ? BOARD_FILES : BLACK_FILES,
        ranks = 'w' === orientation ? WHITE_RANKS : BLACK_RANKS,
        lastMove = (function getLastMove() {
          return game.history({ verbose: !0 }).at(-1) ?? null;
        })();
      boardElement.classList.toggle('is-locked', !canUseBoard());
      for (const rank of ranks)
        for (const file of files) {
          const square = toSquare(file, rank),
            piece = game.get(square),
            button = document.createElement('button');
          if (
            ((button.type = 'button'),
            (button.className = getSquareClasses(square, lastMove)),
            (button.dataset.square = square),
            (button.disabled = !canUseBoard()),
            button.setAttribute('role', 'gridcell'),
            button.setAttribute('aria-label', getSquareLabel(square, piece)),
            file === files[0] && button.appendChild(createCoordinate('rank', String(rank))),
            rank === ranks[ranks.length - 1] &&
              button.appendChild(createCoordinate('file', file)),
            piece)
          ) {
            const pieceElement = document.createElement('span');
            ((pieceElement.className = 'piece'),
              (pieceElement.textContent = PIECE_GLYPHS[piece.color][piece.type]),
              button.appendChild(pieceElement));
          }
          fragment.appendChild(button);
        }
      boardElement.replaceChildren(fragment);
    })(),
    renderStatus(),
    renderAiCandidateDebug(),
    (function renderMoveList() {
      const history = game.history({ verbose: !0 }),
        fragment = document.createDocumentFragment();
```

```
        if (0 === history.length) {
          const empty = document.createElement('p');
          return (
            (empty.className = 'move-empty'),
            (empty.textContent = '아직 둔 수가 없습니다.'),
            fragment.appendChild(empty),
            void moveListElement.replaceChildren(fragment)
          );
        }
        for (let index = 0; index < history.length; index += 2) {
          const row = document.createElement('div'),
            moveNumber = document.createElement('span'),
            whiteMove = document.createElement('span'),
            blackMove = document.createElement('span');
          ((row.className = 'move-row'),
            (moveNumber.className = 'move-number'),
            (whiteMove.className = 'move-san'),
            (blackMove.className = 'move-san'),
            (moveNumber.textContent = `${Math.floor(index / 2) + 1}.`),
            (whiteMove.textContent = history[index]?.san ?? ''),
            (blackMove.textContent = history[index + 1]?.san ?? ''),
            row.append(moveNumber, whiteMove, blackMove),
            fragment.appendChild(row));
        }
        (moveListElement.replaceChildren(fragment),
          (moveListElement.scrollTop = moveListElement.scrollHeight));
      })(),
      (function renderPromotionDialog() {
        if (
          ((promotionOverlay.hidden = null === pendingPromotion),
          promotionChoices.replaceChildren(),
          pendingPromotion)
        )
          for (const piece of PROMOTION_PIECES) {
            const button = document.createElement('button');
            ((button.className = 'promotion-button'),
              (button.type = 'button'),
              (button.dataset.promotion = piece),
              button.setAttribute('aria-label', `${pieceName(piece)} 승격`),
              (button.textContent = PIECE_GLYPHS[game.turn()][piece]),
              promotionChoices.appendChild(button));
          }
      })());
    }
    function renderClocks() {
      const bottomColor = getBoardOrientation(),
        topClockElement = 'w' === oppositeColor(bottomColor) ? whiteClockElement : blackClockElement,
        bottomClockElement = 'w' === bottomColor ? whiteClockElement : blackClockElement;
      ((clockDisplayElement.children[0] === topClockElement &&
        clockDisplayElement.children[1] === bottomClockElement) ||
        clockDisplayElement.replaceChildren(topClockElement, bottomClockElement),
        renderClockPosition('w', bottomColor),
        renderClockPosition('b', bottomColor),
        renderClock('w', whiteClockElement, whiteClockTimeElement),
        renderClock('b', blackClockElement, blackClockTimeElement));
    }
    function renderClockPosition(color, bottomColor) {
      const clockElement = 'w' === color ? whiteClockElement : blackClockElement,
        isBottom = color === bottomColor;
      (clockElement.classList.toggle('is-bottom', isBottom),
        clockElement.classList.toggle('is-top', !isBottom));
    }
    function renderClock(color, clockElement, timeElement) {
      const remainingMs = clockMs[color],
        isActive = activeClockColor === color && null === timedOutColor,
        isTimedOut = timedOutColor === color;
      (clockElement.classList.toggle('is-active', isActive),
        clockElement.classList.toggle('is-warning', remainingMs <= 3e4),
        clockElement.classList.toggle('is-danger', remainingMs <= 1e4),
        clockElement.classList.toggle('is-timeout', isTimedOut),
        (timeElement.textContent = (function formatClockTime(milliseconds) {
```

```
    const safeMilliseconds = Math.max(0, milliseconds);
    if (safeMilliseconds < 1e4)
      return `0:${(safeMilliseconds / 1e3).toFixed(1).padStart(4, '0')}`;
    const totalSeconds = Math.ceil(safeMilliseconds / 1e3);
    return `${Math.floor(totalSeconds / 60)}:${String(totalSeconds % 60).padStart(2, '0')}`;
  })(remainingMs)));
}
function renderStatus() {
  const historyLength = game.history().length;
  ((statusElement.textContent = (function getStatusText() {
    return game.isCheckmate()
      ? `${colorName(oppositeColor(game.turn()))} 체크메이트 승리`
      : game.isStalemate()
        ? '스테일메이트 무승부'
        : game.isInsufficientMaterial()
          ? '기물 부족 무승부'
          : game.isThreefoldRepetition()
            ? '3회 반복 무승부'
            : game.isDrawByFiftyMoves()
              ? '50수 규칙 무승부'
              : game.isDraw()
                ? '무승부'
                : timedOutColor
                  ? [
                      `${colorName(timedOutColor)} 시간 초과`,
                      `${colorName(oppositeColor(timedOutColor))} 승리`,
                    ].join(' · ')
                  : aiThinking
                    ? 'AI 계산 중'
                    : game.isCheck()
                      ? `${colorName(game.turn())} 체크`
                      : `${colorName(game.turn())} 차례`;
  })()),
    (turnMetaElement.textContent = colorName(game.turn())),
    (modeMetaElement.textContent =
      'local' === mode
        ? '로컬 2인'
        : ['AI 상대', `인간 ${colorName(humanColor)}`, presetName(aiSettings.preset)].join(
            ' · ',
          )),
    (moveMetaElement.textContent = [`${game.moveNumber()}수`, `${historyLength} half-move`].join(
      ' · ',
    )));
}
function renderAiCandidateDebug() {
  if ('ai' !== mode)
    return (
      (aiCandidatePanelElement.hidden = !0),
      (aiCandidateMetaElement.textContent = ''),
      (aiInfoStatusElement.textContent = ''),
      (aiInfoPresetElement.textContent = ''),
      (aiInfoHumanElement.textContent = ''),
      (aiInfoDepthElement.textContent = ''),
      (aiInfoNodesElement.textContent = ''),
      (aiInfoScoreElement.textContent = ''),
      void aiCandidateListElement.replaceChildren()
    );
  const debug = aiProgress?.debug;
  if (
    ((aiCandidatePanelElement.hidden = !1),
    (aiCandidateMetaElement.textContent = aiProgress
      ? `경과 ${formatSeconds(aiProgress.elapsedMs)}`
      : ''),
    (aiInfoStatusElement.textContent = (function getAiInfoStatusText() {
      return game.isGameOver() || null !== timedOutColor
        ? '대국 종료'
        : aiThinking
          ? aiProgress
            ? '계산 중'
            : '계산 준비 중'
          : game.turn() === humanColor
```

```
        ? '대기 중'
        : '응수 대기';
    })()),
    (aiInfoPresetElement.textContent = presetName(aiSettings.preset)),
    (aiInfoHumanElement.textContent = colorName(humanColor)),
    (aiInfoDepthElement.textContent = aiProgress ? String(aiProgress.depthReached) : '-'),
    (aiInfoNodesElement.textContent = aiProgress
      ? `${(function formatNodes(nodes) {
          return nodes >= 1e6 ? `${Math.round(nodes / 1e3)}k` : String(nodes);
        })(aiProgress.nodes)} nodes`
      : '-'),
    (aiInfoScoreElement.textContent = aiProgress ? formatDebugScore(aiProgress.score) : '-'),
    !debug || 0 === debug.candidates.length)
  )
    return void aiCandidateListElement.replaceChildren();
  aiCandidateMetaElement.textContent = [
    aiCandidateMetaElement.textContent,
    `best ${formatDebugScore(debug.bestScore)}`,
    `창 ${Math.round(debug.windowCp)}`,
    `p ${debug.ticketPower.toFixed(2)}`,
  ].join('  ');
  const fragment = document.createDocumentFragment();
  for (const candidate of debug.candidates) fragment.appendChild(createAiCandidateRow(candidate));
  aiCandidateListElement.replaceChildren(fragment);
}
function createAiCandidateRow(candidate) {
  const row = document.createElement('div');
  ((row.className = 'ai-candidate-row'),
    row.classList.toggle('is-selected', candidate.selected),
    row.classList.toggle('is-principal', candidate.principal),
    row.setAttribute('role', 'row'));
  const sanCell = createAiCandidateCell('ai-candidate-san', candidate.san);
  return (
    candidate.selected && sanCell.appendChild(createAiCandidateBadge('선택')),
    candidate.principal && sanCell.appendChild(createAiCandidateBadge('최선')),
    row.append(
      sanCell,
      createAiCandidateCell('ai-candidate-score', formatDebugScore(candidate.score)),
      createAiCandidateCell(
        'ai-candidate-weight',
        (function formatDebugWeight(weight) {
          return 0 === weight
            ? '0'
            : weight < 0.001
              ? '<0.001'
              : weight.toFixed(3).replace(/0+$/, '').replace(/\.$/, '');
        })(candidate.tickets),
      ),
      createAiCandidateCell(
        'ai-candidate-probability',
        (function formatDebugProbability(probability) {
          const percent = 100 * probability;
          return 0 === percent ? '0%' : percent < 0.1 ? '<0.1%' : `${percent.toFixed(1)}%`;
        })(candidate.probability),
      ),
    ),
    row
  );
}
function createAiCandidateCell(className, text) {
  const cell = document.createElement('span');
  return (
    (cell.className = className),
    cell.setAttribute('role', 'cell'),
    (cell.textContent = text),
    cell
  );
}
function createAiCandidateBadge(text) {
  const badge = document.createElement('span');
  return ((badge.className = 'ai-candidate-badge'), (badge.textContent = text), badge);
```

```
  }
  function getSquareClasses(square, lastMove) {
    const classes = ['board-square', 'light' === game.squareColor(square) ? 'is-light' : 'is-dark'],
      destinationMove = legalMoves.find((move) => move.to === square);
    return (
      selectedSquare === square && classes.push('is-selected'),
      destinationMove && classes.push(destinationMove.isCapture() ? 'is-capture' : 'is-legal'),
      !lastMove ||
        (lastMove.from !== square && lastMove.to !== square) ||
        classes.push('is-last-move'),
      (function isCheckedKing(square) {
        const piece = game.get(square);
        return Boolean(
          piece && 'k' === piece.type && piece.color === game.turn() && game.isCheck(),
        );
      })(square) && classes.push('is-check'),
      classes.join(' ')
    );
  }
  function getBoardOrientation() {
    return 'ai' === mode ? humanColor : game.turn();
  }
  function canUseBoard() {
    return !(
      game.isGameOver() ||
      null !== timedOutColor ||
      aiThinking ||
      pendingPromotion ||
      ('local' !== mode && game.turn() !== humanColor)
    );
  }
  (boardElement.addEventListener('click', (event) => {
    const button = (function getSquareButton(target, boardElement) {
      if (!(target instanceof Element)) return null;
      const button = target.closest('.board-square');
      return button && boardElement.contains(button) ? button : null;
    })(event.target, boardElement);
    button &&
      (function isSquare(value) {
        return 'string' == typeof value && SQUARE_SET.has(value);
      })(button.dataset.square) &&
      (function handleSquareClick(square) {
        if (!canUseBoard()) return;
        const piece = game.get(square);
        if (selectedSquare) {
          const movesToDestination = legalMoves.filter((move) => move.to === square);
          return movesToDestination.length > 0
            ? movesToDestination.some((move) => move.isPromotion())
              ? ((pendingPromotion = {
                  from: selectedSquare,
                  to: square,
                  moves: movesToDestination,
                }),
                void render())
              : void playMove({ from: selectedSquare, to: square })
            : piece?.color === game.turn()
              ? void selectSquare(square)
              : (clearSelection(), void render());
        }
        piece?.color === game.turn() && selectSquare(square);
      })(button.dataset.square);
  }),
    modeButtons.forEach((button) => {
      button.addEventListener('click', () => {
        const nextMode = button.dataset.mode;
        (function isGameMode(value) {
          return 'local' === value || 'ai' === value;
        })(nextMode) &&
          nextMode !== mode &&
          ((mode = nextMode), resetGame());
      });
```

```
  });
  colorButtons.forEach((button) => {
    button.addEventListener('click', () => {
      const nextColor = button.dataset.color;
      (function isColor(value) {
        return 'w' === value || 'b' === value;
      })(nextColor) &&
        nextColor !== humanColor &&
        ((humanColor = nextColor), resetGame());
    });
  });
  presetButtons.forEach((button) => {
    button.addEventListener('click', () => {
      const preset = button.dataset.preset;
      (function isAiPreset(value) {
        return 'easy' === value || 'normal' === value || 'hard' === value;
      })(preset) && updateAiSettings({ ...AI_PRESETS[preset] });
    });
  });
  clockPresetButtons.forEach((button) => {
    button.addEventListener('click', () => {
      const preset = button.dataset.clockPreset;
      (function isClockPreset(value) {
        return (
          '1/1' === value ||
          '3/2' === value ||
          '5/3' === value ||
          '10/5' === value ||
          '15/10' === value
        );
      })(preset) &&
        preset !== clockPreset &&
        ((clockPreset = preset), resetGame());
    });
  });
  aiDepthInput.addEventListener('input', () => {
    var value;
    updateAiSettings({
      ...aiSettings,
      maxDepth:
        ((value = aiDepthInput.valueAsNumber), Math.min(5, Math.max(1, Math.round(value)))),
    });
  });
  aiTimeInput.addEventListener('input', () => {
    updateAiSettings({ ...aiSettings, timeLimitMs: aiTimeInput.valueAsNumber });
  });
  newGameButton.addEventListener('click', resetGame),
  undoButton.addEventListener('click', function undoLastTurn() {
    (cancelAiMove(),
      (pendingPromotion = null),
      0 !== game.history().length
        ? ('ai' === mode && game.turn() === humanColor ? (game.undo(), game.undo()) : game.undo(),
          clearSelection(),
          resetClockState(),
          restartClockForCurrentTurn(),
          render())
        : render());
  }),
  promotionChoices.addEventListener('click', (event) => {
    const target = event.target;
    if (!(target instanceof Element)) return;
    const button = target.closest('[data-promotion]'),
      promotion = button?.dataset.promotion;
    (function isPromotionPiece(value) {
      return 'q' === value || 'r' === value || 'b' === value || 'n' === value;
    })(promotion) &&
      (function finishPromotion(promotion) {
        if (!pendingPromotion) return;
        if (!pendingPromotion.moves.find((move) => move.promotion === promotion)) return;
        const { from: from, to: to } = pendingPromotion;
        ((pendingPromotion = null), playMove({ from: from, to: to, promotion: promotion }));
```

```
843 ┆ ····})(promotion);
844 ┆ ···}),
845 ┆ ···resetClockState(),
846 ┆ ···restartClockForCurrentTurn(),
847 ┆ ···render());
848 ┆ })();
849 ┆
```

==== style.css (14083 B / 3bb92984b4f4) ====

```
  1 ┆ *,
  2 ┆ *:before,
  3 ┆ *:after {
  4 ┆ ··box-sizing: border-box;
  5 ┆ }
  6 ┆ html {
  7 ┆ ··min-width: 1280px;
  8 ┆ ··min-height: 100%;
  9 ┆ ··background: var(--page);
 10 ┆ ··color-scheme: light;
 11 ┆ }
 12 ┆ body {
 13 ┆ ··min-width: 1280px;
 14 ┆ ··min-height: 100vh;
 15 ┆ ··min-height: 100dvh;
 16 ┆ ··margin: 0;
 17 ┆ ··background: var(--page);
 18 ┆ ··font-family:
 19 ┆ ····Inter,
 20 ┆ ····ui-sans-serif,
 21 ┆ ····system-ui,
 22 ┆ ····-apple-system,
 23 ┆ ····BlinkMacSystemFont,
 24 ┆ ····Segoe UI,
 25 ┆ ····sans-serif;
 26 ┆ ··text-rendering: geometricPrecision;
 27 ┆ }
 28 ┆ button {
 29 ┆ ··font: inherit;
 30 ┆ }
 31 ┆ button:not(:disabled) {
 32 ┆ ··cursor: pointer;
 33 ┆ }
 34 ┆ button:disabled {
 35 ┆ ··cursor: not-allowed;
 36 ┆ }
 37 ┆ :root {
 38 ┆ ··--app-padding: 28px;
 39 ┆ ··--viewport-height: 100vh;
 40 ┆ ··--page: oklch(95% 0.02 85deg / 1);
 41 ┆ ··--ink: oklch(26% 0.01 250deg / 1);
 42 ┆ ··--muted: oklch(54% 0.01 185deg / 1);
 43 ┆ ··--line: oklch(81% 0.02 80deg / 1);
 44 ┆ ··--panel: oklch(99% 0.01 85deg / 1);
 45 ┆ ··--button: oklch(33% 0.03 205deg / 1);
 46 ┆ ··--button-text: oklch(100% 0 0deg / 1);
 47 ┆ ··--accent: oklch(55% 0.15 35deg / 1);
 48 ┆ ··--light-square: oklch(89% 0.05 90deg / 1);
 49 ┆ ··--dark-square: oklch(59% 0.08 150deg / 1);
 50 ┆ ··--last-move: oklch(80% 0.12 90deg / 1);
 51 ┆ ··--legal: oklch(43% 0.12 260deg / 1);
 52 ┆ ··--capture: oklch(52% 0.15 25deg / 1);
 53 ┆ ··--shadow: 0 18px 42px oklch(24% 0.01 75deg / 0.18);
 54 ┆ ··--clear-color: oklch(0% 0 0deg / 0);
 55 ┆ ··--panel-soft: oklch(99% 0.01 85deg / 0.68);
 56 ┆ ··--panel-raised: oklch(99% 0.01 85deg / 0.74);
 57 ┆ ··--panel-strong: oklch(99% 0.01 85deg / 0.88);
 58 ┆ ··--board-edge: oklch(33% 0.03 155deg / 1);
 59 ┆ ··--last-move-strong: oklch(80% 0.12 90deg / 0.76);
 60 ┆ ··--last-move-soft: oklch(80% 0.12 90deg / 0.68);
 61 ┆ ··--legal-marker: oklch(43% 0.12 260deg / 0.42);
 62 ┆ ··--capture-marker: oklch(52% 0.15 25deg / 0.74);
```

```
  --piece-shadow: oklch(0% 0 0deg / 0.2);
  --active-ring: oklch(100% 0 0deg / 0.3);
  --active-halo: oklch(33% 0.03 205deg / 0.18);
  --warning: oklch(69% 0.15 75deg / 1);
  --line-soft: oklch(81% 0.02 80deg / 0.58);
  --line-firm: oklch(81% 0.02 80deg / 0.62);
  --accent-soft: oklch(55% 0.15 35deg / 0.12);
  --button-soft: oklch(33% 0.03 205deg / 0.08);
  --overlay: oklch(21% 0.01 240deg / 0.58);
}
@supports (height: 100dvh) {
  :root {
    --viewport-height: 100dvh;
  }
}
[hidden] {
  display: none !important;
}
.chess-app {
  --board-min-size: 560px;
  --board-vertical-breathing: 104px;
  --side-columns-width: 628px;
  --grid-gaps-width: 30px;
  --board-size: max(
    var(--board-min-size),
    min(
      calc(
        var(--viewport-height) - var(--app-padding) - var(--app-padding) -
        var(--board-vertical-breathing)
      ),
      calc(
        100vw - var(--app-padding) - var(--app-padding) - var(--side-columns-width) -
        var(--grid-gaps-width)
      )
    )
  );
  --panel-max-height: calc(var(--viewport-height) - var(--app-padding) - var(--app-padding));
  width: 100%;
  min-width: 1280px;
  min-height: var(--viewport-height);
  display: grid;
  grid-template-columns: 230px var(--board-size) 118px 280px;
  gap: 10px;
  align-items: stretch;
  justify-content: center;
  overflow: hidden;
  padding: var(--app-padding);
  background: var(--page);
  color: var(--ink);
}
.board-stage {
  width: var(--board-size);
  align-self: center;
  justify-self: center;
}
.clock-panel {
  width: 100%;
  height: var(--board-size);
  min-width: 0;
  min-height: 0;
  align-self: center;
  justify-self: stretch;
  display: flex;
}
.chess-board {
  width: 100%;
  aspect-ratio: 1;
  display: grid;
  grid-template-columns: repeat(8, minmax(0, 1fr));
  overflow: hidden;
  border: 3px solid var(--board-edge);
```

```
  border-radius: 8px;
  box-shadow: var(--shadow);
  background: var(--board-edge);
}
.board-square {
  position: relative;
  width: 100%;
  aspect-ratio: 1;
  display: grid;
  place-items: center;
  min-width: 0;
  padding: 0;
  border: 0;
  color: var(--ink);
  transition:
    box-shadow 0.14s ease,
    filter 0.14s ease,
    transform 0.14s ease;
}
.board-square:not(:disabled):hover {
  filter: brightness(1.05);
}
.board-square.is-light {
  background: var(--light-square);
}
.board-square.is-dark {
  background: var(--dark-square);
}
.board-square.is-selected {
  box-shadow: inset 0 0 0 4px var(--legal);
}
.board-square.is-last-move {
  box-shadow: inset 0 0 0 4px var(--last-move-strong);
}
.board-square.is-selected.is-last-move {
  box-shadow:
    inset 0 0 0 4px var(--legal),
    inset 0 0 0 8px var(--last-move-soft);
}
.board-square.is-legal:after,
.board-square.is-capture:after {
  position: absolute;
  z-index: 1;
  content: '';
  border-radius: 999px;
  pointer-events: none;
}
.board-square.is-legal:after {
  width: 28%;
  height: 28%;
  background: var(--legal-marker);
}
.board-square.is-capture:after {
  inset: 12%;
  border: 3px solid var(--capture-marker);
}
.board-square.is-check {
  box-shadow: inset 0 0 0 5px var(--capture);
}
.chess-board.is-locked .board-square {
  filter: saturate(0.86);
}
.piece {
  z-index: 2;
  display: block;
  font-size: 4rem;
  font-size: max(4rem, var(--board-size) / 8 * 0.92);
  line-height: 1;
  text-shadow: 0 2px 6px var(--piece-shadow);
  -webkit-user-select: none;
  user-select: none;
```

```
}
.coordinate {
  position: absolute;
  z-index: 3;
  font-size: 0.7rem;
  font-size: max(0.7rem, var(--board-size) / 80);
  font-weight: 800;
  line-height: 1;
  opacity: 0.72;
  pointer-events: none;
}
.coordinate-rank {
  top: 6px;
  left: 7px;
}
.coordinate-file {
  right: 7px;
  bottom: 6px;
}
.control-panel,
.info-panel {
  min-width: 0;
  min-height: 0;
  max-height: var(--panel-max-height);
  display: flex;
  flex-direction: column;
  gap: 14px;
  overflow: auto;
  overscroll-behavior: contain;
  scrollbar-gutter: stable;
}
.control-panel {
  justify-self: end;
}
.info-panel {
  justify-self: start;
}
.control-label {
  margin: 0;
  color: var(--muted);
  font-size: 0.78rem;
  font-weight: 800;
  letter-spacing: 0;
  text-transform: uppercase;
}
.control-group,
.status-panel,
.ai-candidate-panel,
.history-panel {
  display: grid;
  gap: 10px;
  padding-top: 16px;
  border-top: 1px solid var(--line);
}
.history-panel {
  min-height: 0;
  flex: 1;
  grid-template-rows: auto minmax(0, 1fr);
}
.segmented-control {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 4px;
  padding: 4px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-soft);
}
.segmented-control-three {
  grid-template-columns: repeat(3, minmax(0, 1fr));
}
```

```
.clock-preset-control {
  display: grid;
  grid-template-columns: repeat(5, minmax(0, 1fr));
  gap: 4px;
  padding: 4px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-soft);
}
.segmented-button,
.primary-button,
.ghost-button,
.promotion-button {
  min-height: 42px;
  border-radius: 6px;
  font-weight: 800;
  letter-spacing: 0;
}
.segmented-button {
  border: 0;
  background: var(--clear-color);
  color: var(--muted);
}
.segmented-button.is-active {
  background: var(--button);
  color: var(--button-text);
}
.clock-settings .segmented-button {
  min-height: 36px;
  padding-inline: 4px;
  font-size: 0.82rem;
}
.ai-settings {
  gap: 12px;
}
.range-control,
.toggle-control {
  display: grid;
  gap: 8px;
  min-width: 0;
  color: var(--ink);
  font-size: 0.92rem;
  font-weight: 800;
}
.range-control span,
.toggle-control {
  align-items: center;
}
.range-control span {
  display: flex;
  justify-content: space-between;
  gap: 10px;
}
.range-control strong {
  color: var(--accent);
  white-space: nowrap;
}
.range-control input[type='range'] {
  width: 100%;
  accent-color: var(--button);
}
.toggle-control {
  grid-template-columns: auto minmax(0, 1fr);
  padding: 10px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-raised);
}
.toggle-control input {
  width: 18px;
  height: 18px;
```

```
  accent-color: var(--button);
}
.action-row {
  display: grid;
  grid-template-columns: 1fr 1fr;
  gap: 10px;
}
.primary-button,
.ghost-button {
  border: 1px solid var(--button);
}
.primary-button {
  background: var(--button);
  color: var(--button-text);
}
.ghost-button {
  background: var(--clear-color);
  color: var(--button);
}
.primary-button:disabled,
.ghost-button:disabled,
.segmented-button:disabled {
  opacity: 0.46;
}
.game-status {
  min-height: 1.4em;
  color: var(--accent);
  font-size: 1.35rem;
  line-height: 1.2;
}
.clock-display {
  width: 100%;
  height: 100%;
  min-height: 0;
  min-width: 0;
  display: flex;
  flex-direction: column;
  gap: 10px;
  justify-content: space-between;
}
.clock-card {
  --clock-bg: var(--panel-raised);
  --clock-border: var(--line);
  --clock-side: var(--muted);
  --clock-time: var(--ink);
  width: 100%;
  min-block-size: 82px;
  max-block-size: 110px;
  min-width: 0;
  display: grid;
  place-content: center;
  gap: 4px;
  padding: 12px 8px;
  border: 1px solid var(--clock-border);
  border-radius: 8px;
  background: var(--clock-bg);
  text-align: center;
}
#white-clock {
  --clock-bg: oklch(99% 0.01 85deg / 1);
  --clock-border: oklch(81% 0.02 80deg / 1);
  --clock-side: oklch(54% 0.01 185deg / 1);
  --clock-time: oklch(26% 0.01 250deg / 1);
}
#black-clock {
  --clock-bg: oklch(33% 0.03 205deg / 1);
  --clock-border: oklch(25% 0.02 210deg / 1);
  --clock-side: oklch(99% 0.01 85deg / 0.74);
  --clock-time: oklch(99% 0.01 85deg / 1);
}
.clock-card.is-active {
```

```
  border-color: var(--button);
  box-shadow:
    inset 0 0 0 2px var(--active-ring),
    0 0 0 2px var(--active-halo);
}
.clock-card.is-warning .clock-time {
  color: var(--warning);
}
.clock-card.is-danger .clock-time,
.clock-card.is-timeout .clock-time {
  color: var(--capture);
}
.clock-side {
  color: var(--clock-side);
  font-size: 0.72rem;
  font-weight: 800;
  line-height: 1;
}
.clock-time {
  font-variant-numeric: tabular-nums;
  color: var(--clock-time);
  font-size: 1.35rem;
  line-height: 1;
}
.status-grid {
  display: grid;
  grid-template-columns: 1fr;
  gap: 8px;
  margin: 0;
}
.status-grid div {
  min-width: 0;
  padding: 10px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-raised);
}
.status-grid dt,
.status-grid dd {
  margin: 0;
}
.status-grid dt {
  color: var(--muted);
  font-size: 0.72rem;
  font-weight: 800;
}
.status-grid dd {
  margin-top: 4px;
  overflow-wrap: anywhere;
  font-size: 0.92rem;
  font-weight: 800;
}
.history-header {
  display: flex;
  align-items: center;
  justify-content: space-between;
}
.ai-candidate-panel {
  min-height: 0;
}
.ai-info-grid {
  display: grid;
  grid-template-columns: repeat(2, minmax(0, 1fr));
  gap: 6px;
  margin: 0;
}
.ai-info-grid div {
  min-width: 0;
  padding: 8px;
  border: 1px solid var(--line);
  border-radius: 8px;
```

```
  background: var(--panel-raised);
}
.ai-info-grid dt,
.ai-info-grid dd {
  margin: 0;
}
.ai-info-grid dt {
  color: var(--muted);
  font-size: 0.64rem;
  font-weight: 800;
  text-transform: uppercase;
}
.ai-info-grid dd {
  margin-top: 3px;
  overflow: hidden;
  font-size: 0.8rem;
  font-weight: 800;
  line-height: 1.1;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.ai-info-grid dd[id='ai-info-nodes'],
.ai-info-grid dd[id='ai-info-score'],
.ai-info-grid dd[id='ai-info-depth'] {
  font-variant-numeric: tabular-nums;
}
.ai-candidate-header {
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 8px;
}
.ai-candidate-meta {
  min-width: 0;
  overflow: hidden;
  color: var(--muted);
  font-size: 0.68rem;
  font-weight: 800;
  text-align: right;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.ai-candidate-table {
  min-height: 0;
  overflow: hidden;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-raised);
}
.ai-candidate-list {
  max-height: 172px;
  overflow: auto;
}
.ai-candidate-row {
  display: grid;
  grid-template-columns: minmax(3rem, 1.3fr) 2.6rem 3.2rem 3.2rem;
  gap: 6px;
  align-items: center;
  min-width: 0;
  padding: 6px 8px;
  border-bottom: 1px solid var(--line-soft);
  font-size: 0.72rem;
  font-weight: 800;
  line-height: 1.15;
}
.ai-candidate-row:last-child {
  border-bottom: 0;
}
.ai-candidate-head {
  color: var(--muted);
  background: var(--panel-strong);
```

```
  font-size: 0.66rem;
  text-transform: uppercase;
}
.ai-candidate-row.is-selected {
  background: var(--accent-soft);
}
.ai-candidate-row.is-principal:not(.is-selected) {
  background: var(--button-soft);
}
.ai-candidate-san,
.ai-candidate-score,
.ai-candidate-weight,
.ai-candidate-probability {
  min-width: 0;
  overflow: hidden;
  text-overflow: ellipsis;
  white-space: nowrap;
}
.ai-candidate-score,
.ai-candidate-weight,
.ai-candidate-probability {
  text-align: right;
  font-variant-numeric: tabular-nums;
}
.ai-candidate-badge {
  display: inline-block;
  margin-left: 4px;
  padding: 1px 3px;
  border-radius: 4px;
  background: var(--button);
  color: var(--button-text);
  font-size: 0.56rem;
  line-height: 1;
  vertical-align: 1px;
}
.move-list {
  min-height: 0;
  max-height: none;
  overflow: auto;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel-raised);
}
.move-row {
  display: grid;
  grid-template-columns: 3rem minmax(0, 1fr) minmax(0, 1fr);
  gap: 8px;
  padding: 8px 10px;
  border-bottom: 1px solid var(--line-firm);
  font-weight: 800;
}
.move-row:last-child {
  border-bottom: 0;
}
.move-number {
  color: var(--muted);
}
.move-san {
  min-width: 0;
  overflow-wrap: anywhere;
}
.move-empty {
  margin: 0;
  padding: 14px 12px;
  color: var(--muted);
  font-weight: 700;
}
.promotion-overlay {
  position: fixed;
  inset: 0;
  z-index: 10;
```

```
  display: grid;
  place-items: center;
  padding: 20px;
  background: var(--overlay);
}
.promotion-overlay[hidden] {
  display: none;
}
.promotion-dialog {
  width: 360px;
  display: grid;
  gap: 16px;
  padding: 20px;
  border: 1px solid var(--line);
  border-radius: 8px;
  background: var(--panel);
  box-shadow: var(--shadow);
}
.promotion-dialog h2 {
  margin: 0;
  font-size: 1.1rem;
}
.promotion-choices {
  display: grid;
  grid-template-columns: repeat(4, minmax(0, 1fr));
  gap: 8px;
}
.promotion-button {
  min-height: 70px;
  border: 1px solid var(--button);
  background: var(--button-text);
  color: var(--ink);
  font-size: 2.4rem;
  line-height: 1;
}

```